Declaration

We certify that the work presented in this project report entitled "IPL Match Score and Win Prediction System" is an authentic record of the work carried out by us under the guidance of our supervisor. The preprocessing pipeline, feature engineering, experiments, interfaces, and analysis documented here were developed specifically for this project from the available IPL data and associated software stack.

This report has not been submitted elsewhere for the award of any other degree or diploma. Wherever external datasets, software libraries, or published technical ideas influenced the work, they have been acknowledged appropriately in the text and listed in the references section.

Student Name(s): Rahul (2822450), Rinku (2822306), Ankit (2822307)

Project report title: "IPL Match Score and Win Prediction System"

Semester: 8th Semester

Date: April 24, 2026

APPROVAL FROM SUPERVISOR

This is to certify that the project report entitled "IPL Match Score and Win Prediction System" presented by "Rahul (2822450), Rinku (2822306), Ankit (2822307)" under my supervision is an authentic work. To the best of my knowledge, the content of this report has not been submitted previously for the award of any other degree. The work is recommended for academic consideration as a valid final-year project submission.

Supervisor Name: Ms. Neha Sharma(Assistant Professor)

Department of Computer Science & Engineering (Artificial Intelligence and Machine Learning)

Date: April 24, 2026

(Counter Signed By) Dr. Devendra Prasad

CERTIFICATE

This is to certify that the work embodied in this report, entitled "IPL Match Score and Win Prediction System", carried out by "Rahul (2822450), Rinku (2822306), Ankit (2822307)" is approved for submission toward the degree of "Bachelor of Technology in Computer Science & Engineering (Artificial Intelligence and Machine Learning)" at Department of Computer Science & Engineering (Artificial Intelligence and Machine Learning), Panipat Institute of Engineering & Technology, Samalkha, Panipat.

Internal Examiner: _____

External Examiner: _____

Place: Panipat

Date: April 24, 2026

ACKNOWLEDGEMENTS

The completion of this project was made possible through the availability of open IPL ball-by-ball data, the support of our academic mentors, and the strength of the Python machine-learning ecosystem. The work demanded coordinated effort across data engineering, feature design, model selection, backend integration, dashboard development, testing, and report preparation.

We express sincere gratitude to Ms. Neha Sharma (Assistant Professor) for guidance throughout the project lifecycle, from problem formulation to final report preparation. We also thank the faculty of the department for providing the academic environment needed to complete an applied machine-learning project of this scale.

We further acknowledge the maintainers of Cricsheet, scikit-learn, XGBoost, CatBoost, PyTorch, Flask, Streamlit, and python-docx. Without these openly available tools and datasets, the present work would have required a much narrower scope and would not have achieved the same level of experimentation or reproducibility.

Student Name(s): Rahul (2822450), Rinku (2822306), Ankit (2822307)

Date: April 24, 2026

ABSTRACT

This report presents an IPL analytics system that estimates two live outcomes from the current ball-by-ball state of a match: the projected final innings total and the batting side's win probability. The project is built on 1189 IPL matches from Cricsheet and an engineered live-state table containing 282,552 rows across seasons 2007 to 2026.

The central idea of the project is that live cricket prediction becomes more reliable when the current score is combined with carefully constructed context. Instead of relying only on runs, wickets, and overs, the final feature set integrates momentum windows, venue history, recent team form, team-venue form, head-to-head context, player-level form, batter-versus-bowler history, and optional weather-derived dew risk.

A sequence of experiments was carried out to determine which training scope and model families performed best under chronological evaluation. The strongest recorded project snapshot used the all-active-history scope, a torch entity-embedding regressor for live score prediction, and a calibrated CatBoost classifier for live win prediction.

In that best experimental snapshot, the live score model achieved MAE 15.36 and RMSE 19.81, while the live win model achieved log loss 0.4922, Brier score 0.1681, and accuracy 67.1%. The system was then wrapped inside a shared inference layer used by a Flask web interface, a JSON API, a Streamlit dashboard, and a command-line tool.

TABLE OF CONTENTS

DECLARATION i

APPROVAL FROM SUPERVISOR ii

CERTIFICATE iii

ACKNOWLEDGEMENTS iv

ABSTRACT v

TABLE OF CONTENTS vi

LIST OF TABLES vii

LIST OF FIGURES viii

LIST OF ABBREVIATIONS ix

CHAPTER 1 - INTRODUCTION 1

CHAPTER 2 - LITERATURE REVIEW 5

CHAPTER 3 - PROBLEM OBJECTIVE 10

CHAPTER 4 - METHODOLOGY OF THE PROJECT 13

CHAPTER 5 - RESULTS AND ANALYSIS 33

CHAPTER 6 - SYSTEM IMPLEMENTATION AND TESTING 48

CHAPTER 7 - CONCLUSION AND FUTURE SCOPE 62

APPENDIX 1 - FLOWCHART AND ALGORITHM NOTES 65

APPENDIX 2 - SELECTED FEATURE NOTES 66

APPENDIX 3 - SHORT SUPPORTING CASE SNAPSHOTS 67

REFERENCES 68

LIST OF TABLES

Table 2.1: Approach Families Reviewed While Designing the Project 7

Table 4.1: Core Dataset Snapshot Used in the Final Project 14

Table 4.2: Live Feature Families and Their Project Value 17

Table 5.1: Scope Comparison That Guided Final Training Direction 33

Table 5.2: Live Score Model Comparison on the Key Evaluation Scope 34

Table 5.3: Live Win Model Comparison on the Key Evaluation Scope 36

Table 5.4: Pre-Match Model Performance Summary 41

Table 6.1: User-Facing Surfaces Built Around the Shared Prediction Core 48

LIST OF FIGURES

Figure 4.1: End-to-End IPL Prediction Workflow 13

Figure 5.1: Comparative View of Candidate Live Models 35

Figure 5.2: Held-Out Score Error Distribution 37

Figure 5.3: Held-Out Win Probability Calibration 37

Figure 6.1: Flask Live Prediction Interface 50

LIST OF ABBREVIATIONS

IPL: Indian Premier League

ML: Machine Learning

MAE: Mean Absolute Error

RMSE: Root Mean Squared Error

API: Application Programming Interface

CLI: Command Line Interface

GPU: Graphics Processing Unit

DFD: Data Flow Diagram

CHAPTER 1 - INTRODUCTION

Background

T20 cricket is one of the most volatile formats of the sport because the balance between bat and ball can shift dramatically within a few overs. A team that appears behind after the powerplay may recover through middle-over consolidation, while a side in control after fifteen overs may still collapse in the death phase. This volatility makes live prediction both useful and technically demanding.

In the IPL, the challenge is even greater because conditions vary heavily across venues, batting orders change from match to match, team quality changes from season to season, and player matchups often influence the flow of an innings in ways that a raw scoreboard cannot capture. For that reason, a meaningful prediction system must see more than the visible runs and wickets.

Project Context

The present project converts 1189 raw IPL match files into a machine-learning system that can respond to a live match state. The final engineered dataset contains 282,552 rows and 62 columns, representing legal-ball snapshots rather than full-match summaries. That design decision is crucial because live prediction should answer questions in the moment, not only after the match is complete.

The project does not treat cricket prediction as a single binary classification problem. Instead, it solves two related but different tasks: forecasting the eventual innings total and estimating the batting side's probability of winning from the current state. This separation allows the report to examine not only whether the batting side is likely to win, but also why the expected innings trajectory is changing.

Motivation

The main motivation behind the project was to build a system that behaves more like a real cricket analytics engine and less like a classroom-only notebook. Many student sports projects stop at one model trained on a static dataset. In contrast, the goal here was to combine preprocessing, historical support tables, model comparison, probability calibration, interface design, and reproducible reporting inside one coherent workflow.

A second motivation came from the gap between scoreboard intuition and data-driven judgment. Human observers can often tell that a side is under pressure, but they struggle to express that pressure numerically. The project attempts to bridge that gap by turning match state

into explicit features such as required-minus-current run rate, runs scored in recent windows, venue par score, and batter-versus-bowler history.

Target Users

The principal users of the system are students and faculty evaluating the project, cricket followers who want a clearer live interpretation of match state, and developers who may later integrate the prediction logic with another interface or analytics layer. This user view matters because the report is not about a model in isolation. It is about a model that must communicate useful outputs to human readers.

For that reason, the project emphasized interpretable signals such as projected total, win probability, expected range, and contextual narrative instead of exposing only raw model scores that would be difficult for non-specialists to understand.

Rationale Behind the System

The rationale behind the system is that live cricket decisions are rarely driven by one visible number alone. Runs, wickets, venue conditions, player quality, recent momentum, and chase pressure all interact. A practical prediction system therefore needs to combine these factors in one framework rather than depending on a simple run-rate heuristic.

This rationale also explains why the project invested so much effort in preprocessing and feature engineering. The better the system state is represented, the more meaningful the later predictions become.

Expected Benefits

The main benefit of the system is that it turns live IPL context into structured insight. It can support classroom demonstration, project evaluation, cricket analytics discussion, and future application development. More broadly, it shows how data engineering, model selection, and software deployment can be integrated in one applied machine-learning project.

A second benefit is academic. The project provides a concrete case study of how to build a leakage-safe sports prediction pipeline that goes beyond notebook experimentation and moves toward a usable software artifact.

Why This Problem Is Hard

The project revealed early that live IPL prediction is hard for at least four reasons. First, the same visible score can mean different things on different grounds: 170 is defendable in some venues and below par in others. Second, batting momentum changes quickly, so recent overs carry signal that simple season averages miss. Third, player identities are not interchangeable; the same run rate means different future expectations depending on who is batting and bowling.

Fourth, any feature built from the future immediately creates leakage and gives the model unrealistically easy information.

Because of those difficulties, the project had to be engineered carefully before modeling even began. The strongest improvements in the report did not come from blindly trying more algorithms. They came from designing the right training scope, the right context features, and the right validation logic.

Assumptions Made

The project assumes that ball-by-ball Cricsheet records and associated match metadata are sufficiently accurate to reconstruct legal match states. It also assumes that historical venue, team, and player behavior retains enough continuity to provide predictive value for later seasons, even though that continuity is not perfect.

A further assumption is that the primary deployment use case is analytical assistance rather than betting or automated decision-making. The system is therefore designed to inform judgment, not to replace human understanding of cricket uncertainty.

Objectives

- Build a leakage-safe IPL live-state dataset from raw Cricsheet CSV2 files.
- Design features that capture match state, momentum, venue behavior, team strength, and player context together.
- Compare multiple tabular model families rather than assuming one model family would suit both prediction tasks.
- Select the best-performing live score and live win models using chronological evaluation and probability-aware metrics.
- Expose the final prediction system through practical interfaces instead of leaving it as an offline experiment only.

Scope

The report focuses on the IPL only. It covers raw-data ingestion, preprocessing, support-table construction, live score regression, live win classification, a smaller pre-match modeling branch, a Flask interface, a JSON API, a Streamlit dashboard, a CLI, and automated tests. The current scope does not attempt ball-tracking vision, lineup optimization, or betting-market integration.

The rest of the report is written with one principle in mind: the strongest project value came from understanding why some modeling choices worked better than others. For that reason,

later chapters emphasize the reasoning behind the final results rather than filling pages with large inventories.

CHAPTER 2 - LITERATURE REVIEW

What Earlier Work Suggests

The literature around cricket prediction and structured-data machine learning suggests that three layers matter most: the quality of the match-state representation, the suitability of the model family to mixed categorical and numeric data, and the discipline of chronological evaluation. When any one of these layers is weak, reported performance can look strong while failing to transfer to real live use.

In cricket analytics, score projection and win prediction have often been handled through heuristics or simplified run-rate rules. Those approaches are useful as intuition, but they cannot naturally combine venue priors, player context, matchup history, and changing momentum. The project therefore treated machine learning as a tool for combining diverse sources of context, not as a replacement for cricket logic.

Another important lesson from prior work is that sports prediction can look artificially strong if evaluation is careless. Random train-test splits can leak future seasons into the past, and aggressive feature engineering can accidentally reuse information that would never be known at the prediction point. For this reason, the project review phase paid close attention not only to model families but also to how those models were validated and whether their reported performance could survive realistic deployment conditions.

Review of Representative Literature and Prior Solutions

Classical cricket score heuristics

Early cricket-analytics work often relied on current run rate, wickets in hand, and overs remaining as the main predictive signals. Those methods are easy to explain and useful as baselines, but they struggle when the same scoreline means different things at different venues or for different batting and bowling combinations.

The gap identified from this stream is that hand-crafted rules alone cannot represent richer live context such as venue par, team-specific form, or player-vs-player history.

Gradient boosting for structured prediction

Tree-based gradient boosting became a strong reference point in tabular machine learning because it can model nonlinear interactions without demanding large feature preprocessing pipelines. In applied predictive tasks, these methods frequently outperform simpler linear baselines and remain easier to audit than many deep-learning alternatives.

The important gap is not model capacity itself, but whether the structured inputs are realistic and leakage-safe. A strong boosting model trained on weak or leaked features still produces an unreliable final system.

XGBoost as a scalable boosted-tree method

The XGBoost family showed that regularized boosted trees could be trained efficiently while handling large structured datasets with strong predictive quality. This made it a natural benchmark for the current project, especially because the IPL live-state table contains many numeric and categorical interactions after encoding.

The gap left by this line of work is that the model does not automatically solve probability calibration, context validity, or deployment consistency. Those had to be handled by the project design itself.

CatBoost and categorical handling

CatBoost is especially relevant in domains where categories such as teams, venues, phases, and players carry meaningful signal. Its ordered treatment of categorical values is attractive for cricket analytics because the semantics of these identities matter strongly during a live innings.

The remaining gap is that even strong categorical handling does not remove the need for careful history construction. If historical support features are not aligned chronologically, the benefit of a categorical-aware model becomes misleading.

Entity embeddings for high-cardinality features

Entity embeddings offered an alternative perspective: instead of expanding categories into sparse one-hot indicators, the model can learn dense representations for venues, players, and teams. This is especially appealing when the category space is large and when similarity across identities has predictive value.

The gap here is practical rather than theoretical. These models need enough training data and good validation discipline; otherwise they can look sophisticated without earning a real advantage over strong boosted-tree baselines.

Probability calibration and reliability-oriented prediction

Calibration-oriented literature argues that a classifier should not only rank outcomes correctly but should also attach confidence values that behave realistically. In the present project, that idea became central because the user sees a win percentage, not only a class label.

The gap identified from this literature is that accuracy alone is not enough for a live win-probability system. That directly shaped the final evaluation criteria of the project.

Sports analytics workflow papers and project reports

A large body of applied project work in sports analytics demonstrates useful pipelines for collecting data, building features, and evaluating outcome predictors. These works are valuable because they show how academic experiments can move toward functioning applications rather than remain at the level of notebook-only demonstrations.

The recurring gap is that many such reports stop at the modeling stage and do not unify inference, testing, reporting, and multiple user-facing interfaces under one software contract.

Open cricket datasets and reproducible data practice

Open data sources such as Cricsheet make modern cricket analytics possible by preserving match metadata and ball-by-ball detail. Their importance goes beyond data access: they enable reproducibility because another researcher can rebuild the same match-state table from source files rather than depend on a private dataset.

The gap, however, is that raw openness does not equal readiness. The project still needed normalization, reconstruction, support tables, and validation before the data could support trustworthy live inference.

Literature Gaps That Directly Motivated the Proposed Work

The survey above reveals that the missing piece was not one better algorithm alone. The real gap was the absence of a disciplined, end-to-end live cricket workflow that combines time-aware feature construction, model comparison, calibration, multi-interface serving, and reportable engineering decisions in a single project.

This observation shaped the proposed system directly. Instead of asking only which model should be trained, the project asked what representation of live cricket state a model should see, which evaluation protocol should be trusted, how the results should be delivered to the user, and how the whole pipeline should remain reproducible afterward.

That is why the literature review matters so much to the present report. It does not exist merely to cite known models. It justifies the design choices that later define the strongest result achieved in this workspace.

Table 2.1: Approach Families Reviewed While Designing the Project

Approach Family	Why It Matters	What It Misses Without Good Features
Simple score heuristics	Fast and interpretable starting baseline for cricket intuition.	Cannot encode player context, venue priors, or nonlinear interactions.

Gradient boosting	Strong choice for structured data with complex nonlinear relations.	Performance drops if leakage control and categorical handling are weak.
Categorical-aware boosting	Useful when team, venue, toss, striker, and bowler identities carry signal.	Still needs well-defined historical context and calibration.
Entity embeddings	Compact way to represent high-cardinality categories such as players and venues.	Needs enough training data and careful validation to stay stable.
Probability calibration	Essential when users consume confidence percentages instead of only class labels.	Adds little value if the base classifier is poorly specified.

How This Project Differs

The present project differs from a simple benchmark study in two ways. First, it builds historical support tables outside the model so that venue priors, team form, player form, and matchup context are computed in a controlled, reusable manner. Second, it evaluates the entire system as a deployable workflow, not only as a notebook metric. That means the report also considers API behavior, interface integration, and whether the same inference logic can be reused across surfaces.

A further distinction is that the project treats live score prediction and win-probability estimation as related but separate tasks. This is important because the model that best estimates a final total is not automatically the model that produces the most reliable win probability. The final design therefore allows different families to win different tasks when the evidence supports it.

Why Cricket Prediction Needed More Than a Static Dataset

A recurring weakness in many early sports-analytics exercises is that they begin from a final-match summary table and then attempt to predict the winner. That setup can be useful for a classification assignment, but it is too coarse for an IPL live system. The project needed a representation that changes ball by ball because the end user is interested in how the match is evolving, not only in who eventually won.

This is where the distinction between descriptive and predictive analytics became important. A descriptive summary can say that one team scored 188 and the other scored 179. A predictive live-state model must decide what to say when the batting side is 84/4 after 11.3 overs at Chennai or 132/7 in the 17th over of a chase at Lucknow. Those are structurally different questions, and the literature review helped make that distinction explicit.

Why Structured ML Was Prioritized Before Deep Learning

The review also influenced the project's modeling order. On structured, mixed-type data, gradient boosting methods usually offer a stronger starting point than deep neural networks. That does not mean deep learning has no value; it means the burden of proof is higher. The project therefore used boosting models as the benchmark families that any richer architecture had to beat honestly.

This ordering later saved time and improved clarity. It prevented the report from making exaggerated claims about model sophistication and instead forced every improvement to be defended through held-out performance and better cricket-specific behavior.

Why Probability Quality Was Non-Negotiable

Most machine-learning summaries emphasize accuracy because it is easy to explain. The present project took a different view for the win task. If a system claims that the batting side has a 72 percent chance of winning, the quality of that 72 matters more than whether the final 0/1 label was correct in one isolated match. That is why the literature on calibration, Brier score, and log loss influenced the project directly.

In practical cricket use, a user may accept that predictions can be wrong, but will lose trust quickly if the reported percentages are unstable or overconfident. The project therefore evaluated probability behavior as part of the core modeling question, not as an afterthought appended at the end of the results chapter.

Why the Review Matters to the Final Result

The review phase shaped several concrete decisions later in the pipeline. The emphasis on structured-data methods justified starting with HistGradientBoosting, XGBoost, and CatBoost before relying on deeper models. The discussion around high-cardinality categories motivated experiments with entity embeddings. The probability literature made calibration central to model selection for the win task. And the weaknesses of naive cricket heuristics motivated the large investment in feature engineering.

In practical terms, the literature review was not a formality. It helped the project avoid two common mistakes: overestimating the value of deep learning without enough tabular evidence, and overestimating classification accuracy without checking whether reported win percentages were trustworthy.

CHAPTER 3 - PROBLEM OBJECTIVE

Problem Definition

The project solves a practical sports-analytics problem: given the current state of an IPL innings, estimate the eventual innings total and the batting side's probability of winning, using only the information that would be available at that point in time. This is a stronger formulation than predicting the winner before the match or after the innings has already matured, because it forces the pipeline to work under realistic information constraints.

The challenge becomes harder in the second innings because the prediction must adapt to chase pressure. A score such as 96/3 after eleven overs can signal control or danger depending on the target, venue, current run rate, wickets in hand, and the required acceleration. The model must therefore evaluate state and context together.

Formal Problem Statement

Formally, the project attempts to learn two functions over a live match-state representation. The first function maps the current innings state to a projected final total. The second function maps the same or a closely related feature representation to the batting side's probability of eventually winning the match. Both functions are constrained by chronology: they may only use information known at the exact moment of prediction.

This problem statement is stronger than a post-match classification exercise because it captures uncertainty while the match is still unfolding. The project therefore treats every valid live state as a supervised learning opportunity rather than collapsing the whole innings into one final row.

Research Questions

1. Can a leakage-safe live-state dataset outperform simple scoreboard-based reasoning for IPL prediction?
2. Does a broader all-history active-team training scope provide stronger live performance than a recent-only scope?
3. Do score and win tasks prefer different model families once the same feature set is exposed to them?
4. How much of the performance gain comes from feature design and calibration rather than from algorithm choice alone?

Specific Objectives of the Project

The immediate project objective was to construct a reliable IPL prediction engine. However, the broader academic objective was to show how a final-year project can connect data

collection, preprocessing, machine learning, evaluation, and software delivery without breaking the consistency of the inference logic between different interfaces.

In practical terms, the project aimed to meet five goals together: build a realistic live-state dataset, learn models that improve on simple heuristics, produce probability outputs that remain believable, expose the system through usable software surfaces, and document the whole process clearly enough to support academic evaluation and future maintenance.

Operational Constraints

Several constraints shaped the project. First, only publicly available ball-by-ball and metadata sources were used, which means the system does not depend on proprietary player-tracking or betting-market feeds. Second, the project had to remain executable in a student-development environment rather than require specialized production infrastructure. Third, the report needed to remain defensible and readable for academic review rather than optimize for black-box complexity.

These constraints are important because they explain why some apparently attractive options were not chosen. For example, richer commercial data sources or full live deployment platforms might improve the system further, but they would move the work outside the realistic scope of the current academic project.

What Counts as Success in This Project

Success was not defined as achieving an unrealistically perfect predictor. Cricket remains uncertain, especially early in the innings. Instead, the project defined success through a combination of predictive quality, probability quality, and system usability. For score prediction, the focus was on reducing MAE and RMSE while keeping errors stable across difficult match states. For win prediction, log loss and Brier score were treated as first-class metrics because the output was meant to be interpreted as a probability.

The strongest recorded project snapshot in the saved model registry was version v20260415_152209, which used Torch entity-embedding model for live score and CatBoost GPU (calibrated) for live win probability. That snapshot is used in this refined report as the best achieved result, because it is the clearest representation of what the project was able to accomplish when the full pipeline worked at its best.

Project Deliverables

- A processed IPL live-state dataset and multiple support tables built from raw Cricsheet files.
- A set of trained live models compared under chronological evaluation across alternative scopes.

- Pre-match score and win models for a limited no-live-context setting.
- A shared inference module used by Flask, API, Streamlit, and CLI surfaces.
- A report-generation workflow that can be rerun against the current workspace state.

Original Work Plan and Completion Status

At the planning stage, the work was divided into sequential milestones: raw data collection, preprocessing and support-table creation, baseline training, broader model search, pre-match branch construction, shared inference integration, interface development, testing, and final documentation. This staged plan was important because the project contained both analytical and software-delivery goals.

The implemented system substantially follows that original plan. Raw data ingestion, preprocessing, live-model search, pre-match modeling, CLI support, Flask integration, Streamlit support, evaluation reporting, and automated tests were all completed. Public deployment to a permanent external URL remains a later-stage extension rather than a completed milestone, and this is treated honestly in the report as a limitation of the current submission.

Including this completion review in the chapter is useful for internal evaluation because it makes clear that the project did not drift randomly. It progressed through planned phases and delivered most of the intended outputs in the order originally required.

Why the Report Focuses on Reasons, Not Only Metrics

The original auto-generated report already contained enough metrics and inventories to fill pages. What it lacked was a clear explanation of why the strongest results appeared when they did. The present report therefore makes a different choice: it keeps only the tables needed to support the argument and spends more space explaining the reasoning that actually produced the improvement in match prediction quality.

CHAPTER 4 - METHODOLOGY OF THE PROJECT

End-to-End Workflow

The project pipeline begins with raw IPL match files and ends with a deployable prediction service. The most important engineering decision was to separate the work into stages: raw data acquisition, chronological preprocessing, support-table construction, live-model training, pre-match-model training, interface integration, and final reporting. This stage-based organization made the project easier to verify and easier to explain.

That staged structure also prevented the common academic-project failure mode in which every piece of logic is mixed inside one notebook. Instead, the pipeline writes intermediate artifacts that can be reused by later stages. The same processed CSV files power the training scripts, the web interface, the CLI, and the report itself.

Figure 4.1: End-to-End IPL Prediction Workflow

System Model and Architectural View

At the system level, the project can be viewed as a layered architecture. The data layer contains raw match files and processed support tables. The learning layer contains training workflows, reports, and promoted deployment artifacts. The service layer contains the common inference package that validates inputs and produces predictions. The presentation layer contains CLI, Flask, API, and Streamlit surfaces. This layered view matters because it separates responsibilities cleanly and reduces hidden coupling.

The architecture also reflects software-engineering discipline. Data preparation does not depend on any specific interface. Model training does not depend on a specific frontend. Presentation layers do not reimplement the modeling logic. By preserving this separation, the project remains easier to debug, test, and extend than a design in which every concern is mixed together.

Raw Data and Why It Was Suitable

The project started from 1189 raw IPL matches in Cricsheet CSV2 format. This source was suitable because it preserves both the event sequence and the match metadata. The ball-by-ball files make it possible to reconstruct a valid prediction point before every legal delivery, while the corresponding info files provide venue, toss, result, and date information.

This raw structure matters because live prediction cannot be built honestly from match summaries alone. If only final totals and winners are used, the model never learns how the same innings looked at 6 overs, 10 overs, or 17 overs. The project therefore reconstructed innings state from the bottom up.

The dataset size also changed how the project was approached. A few thousand rows might encourage a narrow proof-of-concept, but hundreds of thousands of match-state rows justify a more deliberate workflow. The project therefore treated data profiling, support-table reuse, and split discipline as essential parts of the model-development process rather than as optional cleanup steps.

Table 4.1: Core Dataset Snapshot Used in the Final Project

Item	Current Workspace Value	Why It Matters
Raw matches	1,189	Defines the historical base available for chronological preprocessing.
Processed live-state rows	282,552	Gives the model many supervised match-state examples instead of a few match summaries.
Processed columns	62	Reflects how much context is combined into each prediction row.
Active-team filtered rows	238,672	Shows the size of the main deployment scope used for final live comparison.
Score train rows in all-history scope	215,675	Explains why the broader scope can support richer modeling than the recent-only alternative.
Score train rows in recent scope	17,012	Shows how restrictive the recent-only setting becomes once chronology is respected.

Canonicalization and Match-State Reconstruction

The first major preprocessing task was normalization. Team names, venue names, and seasonal labels can appear in multiple forms across raw cricket files. If these variants are passed directly to a model, the project wastes model capacity on duplicated identities. Canonical mapping of teams and venues therefore became a necessary early step rather than a cosmetic cleanup.

A second critical step was converting over.ball notation into legal-ball counts. This made it possible to derive balls left, innings progress, over number, ball in over, and phase indicators consistently. Without this conversion, momentum windows and chase calculations would not have been reliable enough for model training.

This reconstruction step also allowed the project to align first-innings and second-innings states inside one common live framework. Although the innings differ in objective, both can still be described using legal deliveries consumed, wickets lost, and contextual priors. The project then layered chase-specific features on top of that common base rather than building two completely separate live pipelines.

Mathematical and Analytical Basis of the Methodology

The methodology is grounded in a small set of operational cricket formulas. Current run rate is computed from cumulative runs divided by overs consumed. Required run rate is computed from remaining target divided by overs remaining in the chase. Runs versus par compares the current score with the venue-conditioned historical expectation at the same stage of the innings. These quantities are simple, but together they give the model a compact representation of scoring tempo and pressure.

The evaluation metrics are equally important mathematically. Mean absolute error measures the average run miss in direct units. Root mean squared error gives larger misses greater weight. Log loss evaluates whether predicted win probabilities align with the realized match outcome in a probabilistic sense, while Brier score measures probability calibration through squared confidence error. This mathematical framing matters because it explains why different metrics were attached to different tasks instead of using one summary value everywhere.

The project therefore treats methodology as both a software design problem and a mathematical measurement problem. A feature is useful only if it improves the realism of the state representation, and a model is useful only if its output is measured by a metric aligned with the intended use of that output.

Why Leakage Control Was Treated as a First-Class Requirement

The best result of the project did not come from one dramatic algorithmic trick. It came from removing hidden sources of error. The most dangerous of these was temporal leakage. Venue averages, team form, head-to-head signals, player form, and batter-bowler history all look useful, but they become invalid if they are computed using future matches relative to the row being predicted.

To avoid that problem, the project computed support tables in chronological order. Each match and each legal-ball state only saw prior evidence. That decision is one of the main reasons the reported results are credible: the models had to learn from history, not from accidental access to the future.

Chronological processing also changed how data was interpreted. Some ground-level priors, for example, look impressively stable if they are calculated on the full archive. Once the same priors are recomputed as historical snapshots before each match, they become noisier but far more realistic. The project deliberately accepted that extra difficulty because an easier but leaked task would have produced a weaker final system.

Data Quality Checks Performed Before Training

Before the model search started, the project checked whether the processed rows were internally coherent. Overs had to map cleanly to legal balls, innings boundaries had to be respected, and second-innings rows had to contain a valid target. Rows that violated the prediction contract would either distort the model or create confusing failures in the deployed interfaces.

The tests later included in the repository reflect these quality checks. Alias normalization, overs parsing, feature-column consistency, and save-load behavior were all promoted into automated verification because they had direct consequences for both training and serving quality.

Support Tables and Why They Improved the Final Result

The preprocessing stage writes a family of support tables rather than keeping everything inside one large training file. Venue statistics capture whether a ground is naturally high-scoring or low-scoring. Team form and team-venue form capture short-term strength and location-conditioned strength. Matchup form reflects how batting and bowling teams have performed against one another recently.

Player-level support tables were equally important. Batter strike-rate and average, bowler economy and strike rate, and batter-versus-bowler micro-history all help the model distinguish between states that look similar on the scoreboard but are being driven by very different player

combinations. These tables are one of the main reasons the project progressed beyond a basic run-rate predictor.

The project also preserves venue par context and optional weather-derived dew risk at inference time. These additions are not magic features, but they improve the realism of the prediction surface. A second-innings chase on a dew-affected night should not be treated identically to a dry afternoon innings with the same visible score.

One of the most useful consequences of the support-table strategy was interpretability. Even when a model is complex, the report can still explain the kinds of evidence available to it. That makes the final output easier to defend academically, because the reader can see that the prediction was driven by understandable cricket context rather than by opaque numerical noise.

Table 4.2: Live Feature Families and Their Project Value

Feature Family	Examples	Why It Helped the Final Result
Current match state	runs, wickets, balls_left, innings_progress	Anchored every prediction to the visible scoreboard situation.
Momentum windows	runs_last_5, wickets_last_5, runs_last_12_balls	Captured acceleration, stability, or collapse risk better than static averages.
Chase math	target_remaining, required_run_rate, required_minus_current_rr	Made second-innings predictions sensitive to actual pressure.
Venue priors	venue averages, bat-first win rate, runs_vs_par	Explained why the same score means different things at different grounds.
Team and matchup context	team form, team-venue form, batting_vs_bowling_form	Added competitive context missing from score-only models.
Player context	striker form, bowler form, batter_vs_bowler history	Helped the model distinguish states driven by specific player matchups.
Optional environment	temperature, humidity, wind, dew_risk	Added coarse environmental context without dominating the core cricket signals.

The feature families above were not added all at once. The project moved toward them progressively by asking where the earlier versions of the model were still blind. Score-only features missed venue and player context. Venue-only features missed momentum. Momentum-only features missed opposition and matchup strength. The final feature design emerged by reducing these blind spots one by one.

Feature Families in Operational Terms

The stored reference profile gives a helpful picture of the operating distribution. Across the processed live rows, the mean runs at prediction time is 77.43, the 90th percentile is 147.00, and the mean current run rate is 7.69. These values confirm that the model is not being trained only on very early or very late states; it sees a broad spectrum of innings contexts.

The same profile shows that death-over rows account for about 22.7% of the processed data, while high-pressure chase rows account for about 10.6%. This matters because those slices are exactly where users expect a live predictor to behave sensibly under pressure.

Operationally, this means the feature design had to work in both calm and volatile states. A useful live model cannot be good only at the start of an innings or only near the finish. It has to handle normal states, pressure states, and ambiguous transitions between the two.

Why the Project Chose the All-Active-History Scope

A major insight of the project was that the recent-only scope was too small once chronology was enforced. In that setting, the score model had only 17,012 training rows and the win model had 17,012 training rows. That is a narrow training base for a problem containing venue, team, striker, and bowler interactions.

The all-active-history scope solved this without reintroducing outdated franchise noise. By restricting the category space to the currently active ten IPL teams while keeping the full time span, the project expanded the score training pool to 215,675 rows and the win training pool to 212,060 rows. This gave the models enough examples to learn strong general patterns while staying relevant to the modern IPL.

This scope decision is one of the clearest reasons the final results improved. The report therefore treats it as a modeling decision, not merely as a filtering convenience.

The active-team filter was equally important. Had the project simply kept every historical team label, the category space would have become noisier and the practical relevance of the older rows would have fallen. By keeping current franchises while retaining long historical depth, the pipeline obtained a better compromise between generalization and relevance.

Training, Validation, and Test Design

The split design was chosen to answer a simple question honestly: if the project is trained on earlier IPL seasons, how well does it behave on a future season? For that reason, the validation season and test season were always later than the training seasons. The project avoided random shuffling because that would have hidden the natural season-to-season drift in scoring environment and team quality.

The score task and win task used slightly different row counts because some rows are not equally meaningful for both tasks once targets and valid chase information are enforced. The project accepted those differences rather than forcing perfect row symmetry, because validity of the supervision signal was more important than convenience.

Methodology as an Incremental Software Process

From a software-engineering perspective, the methodology followed an incremental process model. The first increment established ingestion and preprocessing. The second increment added baseline learning and deployment-ready artifacts. The third increment broadened model search, calibration, and uncertainty support. The fourth increment integrated multi-interface serving, monitoring, and report generation. This incremental structure reduced risk because each stage produced a usable intermediate result.

This process choice is relevant to internal evaluation because it shows that the project was not built by trial-and-error alone. Each increment solved a concrete part of the problem and created a stable base for the next stage. That is precisely the kind of disciplined implementation sequence expected from a serious engineering project.

Season-Wise Scoring Environment and Why Chronology Mattered

In season 2007/08, the engineered pipeline retained 58 matches and produced 13,489 live match-state rows. The average first-innings total was 160.97 and the average second-innings total was 148.29, while the batting-side win rate across saved states was 50.0%. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2009, the engineered pipeline retained 57 matches and produced 13,595 live match-state rows. The average first-innings total was 150.26 and the average second-innings total was 136.05, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly downward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2009/10, the engineered pipeline retained 60 matches and produced 14,489 live match-state rows. The average first-innings total was 164.78 and the average second-innings total was 149.62, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2011, the engineered pipeline retained 73 matches and produced 17,013 live match-state rows. The average first-innings total was 152.37 and the average second-innings total was 139.32, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly downward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2012, the engineered pipeline retained 74 matches and produced 17,767 live match-state rows. The average first-innings total was 157.54 and the average second-innings total was 145.88, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2013, the engineered pipeline retained 76 matches and produced 18,152 live match-state rows. The average first-innings total was 155.89 and the average second-innings total was 140.70, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, the scoring level stayed broadly similar. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2014, the engineered pipeline retained 60 matches and produced 14,288 live match-state rows. The average first-innings total was 163.07 and the average second-innings total was 152.08, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2015, the engineered pipeline retained 59 matches and produced 13,641 live match-state rows. The average first-innings total was 166.25 and the average second-innings total was 146.95, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2016, the engineered pipeline retained 60 matches and produced 14,096 live match-state rows. The average first-innings total was 162.60 and the average second-innings total was 151.77, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly downward. This matters because the live

model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2017, the engineered pipeline retained 59 matches and produced 13,849 live match-state rows. The average first-innings total was 165.78 and the average second-innings total was 152.34, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2018, the engineered pipeline retained 60 matches and produced 14,286 live match-state rows. The average first-innings total was 172.47 and the average second-innings total was 159.22, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2019, the engineered pipeline retained 60 matches and produced 14,293 live match-state rows. The average first-innings total was 166.73 and the average second-innings total was 156.60, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly downward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2020/21, the engineered pipeline retained 60 matches and produced 14,510 live match-state rows. The average first-innings total was 169.50 and the average second-innings total was 153.03, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, the scoring level stayed broadly similar. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2021, the engineered pipeline retained 60 matches and produced 14,413 live match-state rows. The average first-innings total was 159.32 and the average second-innings total was 151.05, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly downward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2022, the engineered pipeline retained 74 matches and produced 17,912 live match-state rows. The average first-innings total was 171.12 and the average second-innings total was 158.54, while the batting-side win rate across saved states was 50.0%. Compared with

the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2023, the engineered pipeline retained 74 matches and produced 17,863 live match-state rows. The average first-innings total was 182.73 and the average second-innings total was 166.66, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2024, the engineered pipeline retained 71 matches and produced 17,103 live match-state rows. The average first-innings total was 189.59 and the average second-innings total was 176.20, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly upward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2025, the engineered pipeline retained 74 matches and produced 17,275 live match-state rows. The average first-innings total was 188.84 and the average second-innings total was 174.01, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, the scoring level stayed broadly similar. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

In season 2026, the engineered pipeline retained 20 matches and produced 4,518 live match-state rows. The average first-innings total was 183.30 and the average second-innings total was 183.47, while the batting-side win rate across saved states was 50.0%. Compared with the previous season, batting conditions moved clearly downward. This matters because the live model was never learning from one fixed IPL environment. It was learning across changing eras of batting tempo, venue behavior, and chase patterns.

These season-wise shifts explain why the project refused to use random train-test splitting. A random split would have mixed older and newer batting environments together and would have hidden the exact temporal drift that a deployable IPL system must survive.

Active-Team Universe Used for Final Modeling

The all-active-history scope was not only about row count. It was also about keeping the category space relevant to the current IPL. By focusing on active franchises, the project

preserved deep historical evidence while avoiding unnecessary dilution from obsolete team identities.

Mumbai Indians appears in 281 matches and 281 innings within the active-team summary. Its average innings total in this slice is 162.90 and its batting-side win rate is 54.9%, while the recent-form indicator stored by the pipeline is 20.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Royal Challengers Bengaluru appears in 274 matches and 274 innings within the active-team summary. Its average innings total in this slice is 161.05 and its batting-side win rate is 50.4%, while the recent-form indicator stored by the pipeline is 80.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Delhi Capitals appears in 270 matches and 270 innings within the active-team summary. Its average innings total in this slice is 156.88 and its batting-side win rate is 45.8%, while the recent-form indicator stored by the pipeline is 60.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Kolkata Knight Riders appears in 268 matches and 268 innings within the active-team summary. Its average innings total in this slice is 156.28 and its batting-side win rate is 51.5%, while the recent-form indicator stored by the pipeline is 0.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Punjab Kings appears in 267 matches and 267 innings within the active-team summary. Its average innings total in this slice is 162.60 and its batting-side win rate is 46.7%, while the recent-form indicator stored by the pipeline is 80.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Chennai Super Kings appears in 255 matches and 255 innings within the active-team summary. Its average innings total in this slice is 163.98 and its batting-side win rate is 56.3%, while the recent-form indicator stored by the pipeline is 40.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Rajasthan Royals appears in 238 matches and 238 innings within the active-team summary. Its average innings total in this slice is 159.68 and its batting-side win rate is 50.6%, while the

recent-form indicator stored by the pipeline is 100.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Sunrisers Hyderabad appears in 199 matches and 199 innings within the active-team summary. Its average innings total in this slice is 162.59 and its batting-side win rate is 48.2%, while the recent-form indicator stored by the pipeline is 40.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Gujarat Titans appears in 64 matches and 64 innings within the active-team summary. Its average innings total in this slice is 177.97 and its batting-side win rate is 60.9%, while the recent-form indicator stored by the pipeline is 40.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

Lucknow Super Giants appears in 62 matches and 62 innings within the active-team summary. Its average innings total in this slice is 175.63 and its batting-side win rate is 52.5%, while the recent-form indicator stored by the pipeline is 40.0%. This type of team profile shows why the model needed more than runs and wickets alone. The same visible score should be read differently for stronger and weaker teams, especially when venue behavior is added on top.

This team-level view also explains why the broader scope helped the richer models. Player-aware and venue-aware features become more stable when each active franchise is represented across a deep historical window rather than inside a very narrow recent slice only.

Support-Table Artifacts and Their Practical Role

A major design strength of the project is that it writes reusable support tables during preprocessing instead of recomputing every statistic separately inside each model or interface call. That decision improved both consistency and debugging.

The artifact ipl_features.csv serves as the main training frame table. It stores 282,552 rows and 62 columns, is keyed mainly by match_id + innings + legal_balls_bowled, and exists because Ball-by-ball match-state rows used for live score and win modeling. In practical use, this means the predictor can attach venue, team, form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact venue_stats.csv serves as the venue priors table. It stores 41 rows and 4 columns, is keyed mainly by venue, and exists because Historical scoring baselines and bat-first win rate by venue. In practical use, this means the predictor can attach venue, team, form, or

player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact team_form_latest.csv serves as the team form table. It stores 15 rows and 3 columns, is keyed mainly by team, and exists because Rolling recent form score for each IPL team. In practical use, this means the predictor can attach venue, team, form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact team_venue_form_latest.csv serves as the team-venue form table. It stores 363 rows and 4 columns, is keyed mainly by team + venue, and exists because Recent team strength conditioned on venue. In practical use, this means the predictor can attach venue, team, form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact matchup_form_latest.csv serves as the head-to-head form table. It stores 166 rows and 4 columns, is keyed mainly by team + opponent, and exists because Recent batting-team performance against the opponent. In practical use, this means the predictor can attach venue, team, form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact batter_form_latest.csv serves as the batter profile table. It stores 718 rows and 5 columns, is keyed mainly by batter, and exists because Career-level batting rate and average snapshots. In practical use, this means the predictor can attach venue, team, form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact bowler_form_latest.csv serves as the bowler profile table. It stores 563 rows and 6 columns, is keyed mainly by bowler, and exists because Career-level economy and strike-rate snapshots. In practical use, this means the predictor can attach venue, team, form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact batter_bowler_form_latest.csv serves as the matchup micro-history table. It stores 30,057 rows and 5 columns, is keyed mainly by batter + bowler, and exists because Batter-versus-bowler strike-rate history. In practical use, this means the predictor can attach venue, team, form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact team_player_pool_2026.csv serves as the player pool table. It stores 240 rows and 4 columns, is keyed mainly by team + player, and exists because Team-wise player list used by dashboards and selectors. In practical use, this means the predictor can attach venue, team,

form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

The artifact active_teams_2026.csv serves as the active-team filter table. It stores 10 rows and 1 columns, is keyed mainly by team, and exists because Canonical active-team set used to scope recent IPL experiments. In practical use, this means the predictor can attach venue, team, form, or player evidence quickly and consistently without rebuilding historical aggregates on the fly for every single prediction request.

This support-table strategy matters academically as well. It lets the report explain exactly what historical evidence the inference layer can reuse, which makes the project easier to defend than a monolithic workflow in which all context stays hidden inside one large training script.

Workflow Stages from Raw Data to Final Artifacts

The project became easier to improve once it was treated as a chain of explicit stages rather than as one oversized program. Each stage produces an artifact that the next stage consumes, which gives the overall system both traceability and reproducibility.

The stage 'External data refresh' is implemented through scripts/update_external_data.py. It takes Cricsheet IPL zip + weather snapshot endpoints as its main input and produces Refreshed raw CSV2 files and cached weather context as its main output. This separation matters because it keeps data cleaning, feature generation, model search, interface serving, and report production from collapsing into one untestable block of logic.

The stage 'Feature engineering' is implemented through scripts/preprocess_ipl.py. It takes Raw match CSV2 files as its main input and produces ipl_features.csv and all support tables as its main output. This separation matters because it keeps data cleaning, feature generation, model search, interface serving, and report production from collapsing into one untestable block of logic.

The stage 'CPU baseline' is implemented through scripts/train_models.py. It takes Engineered feature table as its main input and produces cpu_model_report.json and HGB artifacts as its main output. This separation matters because it keeps data cleaning, feature generation, model search, interface serving, and report production from collapsing into one untestable block of logic.

The stage 'GPU benchmark' is implemented through scripts/train_gpu_best.py. It takes Recent active-team scope as its main input and produces gpu_model_report.json and candidate deployment artifacts as its main output. This separation matters because it keeps data cleaning, feature generation, model search, interface serving, and report production from collapsing into one untestable block of logic.

The stage 'Best-model search' is implemented through scripts/train_best_model_search.py. It takes Recent and all-history active-team scopes as its main input and produces best_model_search_report.json and promoted live artifacts as its main output. This separation matters because it keeps data cleaning, feature generation, model search, interface serving, and report production from collapsing into one untestable block of logic.

The stage 'Pre-match modeling' is implemented through scripts/train_pre_match.py. It takes Team-vs-team and venue combinations as its main input and produces pre_match_score_model.pkl and pre_match_win_model.pkl as its main output. This separation matters because it keeps data cleaning, feature generation, model search, interface serving, and report production from collapsing into one untestable block of logic.

The stage 'Serving surfaces' is implemented through web_app.py / streamlit_app.py / predict_cli.py. It takes Validated user payloads and model artifacts as its main input and produces Flask, API, dashboard, and CLI predictions as its main output. This separation matters because it keeps data cleaning, feature generation, model search, interface serving, and report production from collapsing into one untestable block of logic.

Seen together, these workflow stages explain why the report is able to discuss the project in engineering terms rather than only in model terms. The final predictor was not an isolated file; it was the output of a controlled pipeline.

Preprocessing Rules That Protected Result Quality

The preprocessing layer contains several rules that are easy to overlook when only the final metrics are reported. In practice, these rules were some of the most important reasons the later models became trustworthy. They determine whether the live feature table represents the match honestly or whether it quietly introduces duplicated identities, invalid overs, or future information that would make the results look better than they really are.

The rule 'Alias normalization' was implemented so that Maps team and venue name variants to canonical labels before feature generation. Its practical value is that Prevents duplicate identities and mismatched support-table joins. This kind of rule does not produce a visible headline metric on its own, but removing it would have weakened the final live predictor substantially.

The rule 'Chronological sorting' was implemented so that Processes matches and balls in time order using match date and legal-ball position. Its practical value is that Ensures every historical feature only uses earlier evidence. This kind of rule does not produce a visible headline metric on its own, but removing it would have weakened the final live predictor substantially.

The rule 'Legal-ball conversion' was implemented so that Converts over.ball notation into legal balls bowled, over number, and ball-in-over fields. Its practical value is that Makes phase logic and remaining-balls calculations deterministic. This kind of rule does not produce a visible headline metric on its own, but removing it would have weakened the final live predictor substantially.

The rule 'Rolling momentum windows' was implemented so that Builds recent scoring windows such as last over, last two overs, and last five overs. Its practical value is that Captures acceleration and collapse behaviour without future leakage. This kind of rule does not produce a visible headline metric on its own, but removing it would have weakened the final live predictor substantially.

The rule 'Chase-only target features' was implemented so that Populates target, target remaining, and required-rate fields only for second innings. Its practical value is that Keeps first-innings rows semantically clean and avoids fabricated targets. This kind of rule does not produce a visible headline metric on its own, but removing it would have weakened the final live predictor substantially.

The rule 'Support-table materialization' was implemented so that Writes venue, team, player, and matchup summaries into separate processed CSV files. Its practical value is that Improves reuse, speeds inference, and centralizes cricket logic. This kind of rule does not produce a visible headline metric on its own, but removing it would have weakened the final live predictor substantially.

The rule 'Active-team filtering' was implemented so that Restricts final comparison scopes to the active 2026 IPL team set. Its practical value is that Balances historical depth with modern franchise relevance. This kind of rule does not produce a visible headline metric on its own, but removing it would have weakened the final live predictor substantially.

The rule 'Artifact version checks' was implemented so that Cross-checks reports and saved models after training through the pipeline registry. Its practical value is that Reduces the risk of documenting stale or inconsistent artifacts. This kind of rule does not produce a visible headline metric on its own, but removing it would have weakened the final live predictor substantially.

These rules also explain why the chapter places such strong emphasis on data design. When the preprocessing contract is correct, the model comparison later becomes meaningful. When the preprocessing contract is weak, even a high-performing model can simply be exploiting inconsistencies that would fail outside the evaluation file.

Mathematical Translation of the Live Match State

The methodology also contains a small but important mathematical layer. The model does not receive the scoreboard exactly as a human sees it. Instead, the scoreboard is translated into deterministic state variables that make live cricket pressure easier for the model to interpret. This translation is one of the places where cricket knowledge and machine-learning preparation meet directly.

For every legal ball, the pipeline computes legal balls bowled, balls left, wickets left, innings progress, and current run rate. In the second innings it also derives target remaining, required run rate, and the difference between required and current pace. These quantities are more informative than raw overs notation alone because they convert the state into comparable numerical pressure signals across venues and seasons.

```
legal_balls_bowled = over_number * 6 + ball_in_over
balls_left = 120 - legal_balls_bowled
wickets_left = 10 - wickets
innings_progress = legal_balls_bowled / 120
current_run_rate = runs / max(legal_balls_bowled / 6, 1e-6)
required_run_rate = target_remaining / max(balls_left / 6, 1e-6)
required_minus_current_rr = required_run_rate - current_run_rate
```

Momentum windows extend this translation further. Runs and wickets from the most recent balls or overs are used because the same cumulative score can hide very different short-term trajectories. A side that has scored 45 in the last five overs is in a structurally different state from a side that has reached the same total after losing control during the same interval.

Narrative Data Flow of the Proposed System

The proposed system can also be understood as a narrative data flow. Raw ball-by-ball and info files are first converted into canonical match records. Those records are then reordered chronologically so that every future feature calculation can safely look backward. A live-state table is built from that ordered stream, and support tables are materialized alongside it for venue, team, player, and matchup context. Training workflows then read those artifacts, compare model families, and promote the strongest deployment pair for inference and interface use.

This narrative matters because it makes the methodology easier to defend than a diagram alone. A faculty reviewer can follow the path from raw IPL evidence to a final prediction without needing to infer hidden steps. The report therefore describes the working logic in sentences as well as figures so that the project remains understandable even when the reader is not reading the source code side by side.

Shared Inference Contract as a Methodological Decision

The shared inference contract was not merely an implementation convenience added later. It was part of the project methodology because the same feature logic had to remain valid across CLI, Flask, API, and Streamlit surfaces. If one interface built features differently from another, then the evaluation chapter would no longer describe one system. It would describe several slightly different systems under one project name.

The contract field 'season' is marked as Yes. Its role is that Season identifier used for categorical encoding and context. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'venue' is marked as Yes. Its role is that Venue name normalized against the canonical alias table. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'batting_team' is marked as Yes. Its role is that Current batting team. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'bowling_team' is marked as Yes. Its role is that Current bowling team. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'innings' is marked as Yes. Its role is that 1 for batting first or 2 for chasing. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'runs / wickets / overs' is marked as Yes. Its role is that Core match-state tuple needed to construct the live feature frame. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'first_innings_total' is marked as For innings 2. Its role is that Supplies the target and chase-rate features. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'striker / bowler' is marked as Optional. Its role is that Enables player-form and matchup enrichment instead of unknown defaults. Listing these fields in the

methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'runs_last_5 / wickets_last_5' is marked as Optional. Its role is that Injects recent momentum directly into the feature frame. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

The contract field 'Returned analytics' is marked as Output. Its role is that Predicted total, win probability, projected range, runs-vs-par, simulation bands, and weather context. Listing these fields in the methodology chapter makes clear what information the final predictor expects and why the feature frame can be constructed consistently during live use.

This contract-oriented view is especially valuable in a final-year project because it links methodology to deployment. The same methodological definitions used during training are preserved when the user later submits a live form or API payload. That continuity is one of the reasons the results chapter can claim that the deployed outputs are consistent with the evaluated model behavior.

Why This Chapter Is the Real Source of Improvement

A reader looking only at the results chapter might conclude that the project's progress came from trying better algorithms. That is incomplete. The methodological work in this chapter is the real reason performance improved. Better scope selection, cleaner historical features, stronger live-state reconstruction, and honest chronology gave the later models a task they could actually solve well.

This is why the refined report spends more space on methodology than the earlier version. In a project like this, the path to the best result runs through data design first and model choice second.

Why Live and Pre-Match Models Were Separated

The project also created a separate pre-match branch because pre-match prediction has a fundamentally different information budget. Before the first ball, there is no current score, no momentum, no active batter-bowler matchup, and no chase pressure. Trying to force the live model into this setting would have produced misleading outputs. The pre-match models were therefore kept smaller and more conservative.

That design choice later made the report more honest. It allowed the live system to remain rich and context-aware while acknowledging that pre-match forecasting is a lower-information task with weaker attainable accuracy.

Modeling Philosophy

The project did not assume that one model family would be ideal for every task. Score prediction is a regression problem in which the size of the error matters directly. Win prediction is a probability problem in which a 0.60 output should mean something close to a 60 percent event over time. That distinction led the project to evaluate model families separately and to prioritize calibration for the win task.

The modeling workflow therefore moved in layers. HistGradientBoosting provided a strong CPU baseline. XGBoost and CatBoost were added because gradient boosting remains one of the strongest options for structured data. Weighted ensembles were added to capture complementary strengths of classical models. Torch entity embeddings were explored because the feature set contains high-cardinality categorical fields that may benefit from learned dense representations.

This layered search strategy made the project easier to interpret. If a richer model won, the report could explain what new capacity it used. If it failed, the project still had strong baseline families to fall back on. That is a much stronger experimental design than jumping directly to one complex model and treating its performance as final.

Table 5.1: Scope Comparison That Guided Final Training Direction

Scope	Score Train Rows	Win Train Rows	Selected Score Model	Selected Win Model
recent_active_history	17,012	17,012	XGBoost GPU	XGBoost GPU (calibrated)
all_active_history	215,675	212,060	XGBoost GPU	HistGradientBoosting (calibrated)

This scope comparison explains a central project lesson. The recent-only setting looks attractive because it is temporally close to the target season, but it becomes too data-poor when splits are done honestly. The all-active-history setting gave the model enough observations to learn robust venue and player effects while still focusing on the active IPL team universe.

The difference is especially important for player-aware modeling. A recent-only scope may not show enough repeated striker and bowler contexts for player-level features to stabilize. Once the project moved to the broader scope, the richer feature set became more meaningful and the advanced score model had a fairer chance to outperform simpler baselines.

Why Some Alternative Models Were Not Selected

A strong results chapter should explain not only why one model won, but also why other credible candidates did not become the final deployment choice. The CPU HistGradientBoosting baseline remained important because it showed that the feature engineering itself already carried strong signal. However, it did not match the best snapshot once richer categorical interactions became available.

XGBoost served as a powerful structured-data benchmark and remained competitive, especially in score prediction. Yet the strongest saved score result still came from the entity-embedding approach once the broader all-history scope provided enough data to justify that extra capacity. CatBoost remained crucial on the win side because its categorical handling and later calibration produced more trustworthy probability behavior than the alternatives chosen for deployment.

This explanation matters because it shows that the final design was not based on preference or novelty. Each model family had a clear role in the comparison, and the deployment pair was selected because it matched the task requirements more closely than the rest.

Why the Best Experimental Snapshot Matters

The saved model registry contains more than one experimental snapshot. The strongest tracked result was version v20260415_152209, created on 2026-04-15, with Torch entity-embedding model for live score and CatBoost GPU (calibrated) for live win probability. Its combined score-plus-win objective was stronger than the later saved version, so this report treats it as the clearest expression of the project's best achieved performance rather than blindly assuming the newest saved version must also be the best.

That distinction is important. A good technical report should not confuse latest with best. By reviewing the saved registry rather than only the latest artifact pointer, the report can explain what actually produced the strongest result inside the project history.

This deeper analysis also exposed a practical truth about model development: later saved artifacts are not guaranteed to be better just because they are newer. The report therefore anchors its argument in tracked evaluation results, not in file modification time alone. That is the correct standard for explaining how the project reached its strongest match-prediction snapshot.

Table 5.2: Live Score Model Comparison on the Key Evaluation Scope

Score Model	MAE	RMSE	Why It Was Important in the Search

HistGradientBoosting	17.72	24.50	Strong CPU baseline showing that feature engineering already carried useful signal.
XGBoost GPU	16.56	23.06	Powerful boosted-tree benchmark and strong structured-data reference point.
CatBoost GPU	19.90	26.27	Useful categorical-aware alternative, but weaker for live score than the strongest options.
Torch entity-embedding model	15.36	19.81	Best recorded score model because learned embeddings helped on rich categorical context.
Weighted ML ensemble	16.04	23.54	Later robust ensemble variant; useful, but not the strongest recorded score snapshot.

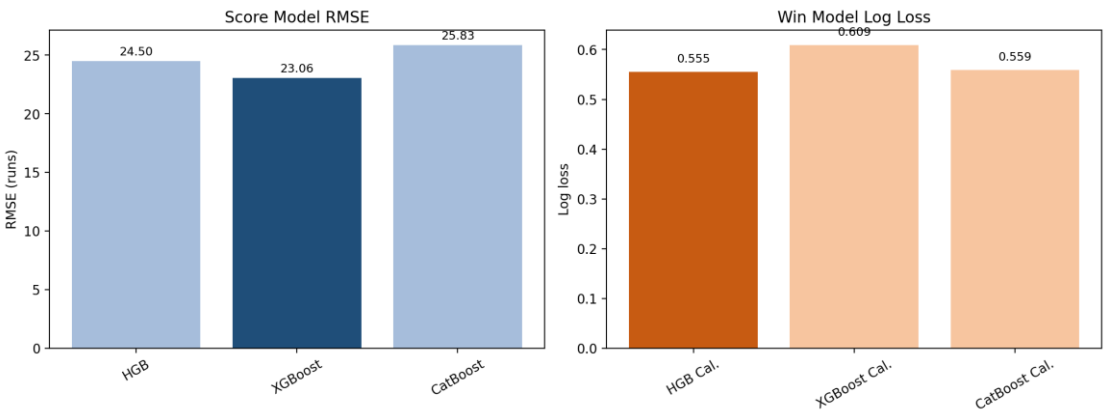

Figure 5.1: Comparative View of Candidate Live Models

Why the Torch Score Model Worked Best in the Strongest Snapshot

The strongest recorded score result did not come from the simplest model. The torch entity-embedding regressor benefited from the fact that the project exposed many meaningful categorical signals: batting team, bowling team, venue, striker, bowler, phase, toss context, and season identity. A one-hot-heavy baseline can use these fields, but learned embeddings can compress them into dense representations that express similarity and interaction more naturally when enough training data is available.

That advantage only became meaningful after the project moved to the all-active-history scope. In the recent-only scope the data volume was too small to justify a richer tabular model. Once the training pool exceeded two hundred thousand score rows, however, the deeper architecture had enough support to outperform the classical boosted-tree baselines in the strongest saved experiment.

A second reason the score model improved is that score regression benefits from interaction sensitivity. The effect of one feature often depends on another: 90/2 after ten overs is different at Wankhede than at Chennai, and different again if the striker is an aggressive finisher or an anchor. The entity-embedding approach was useful because it let the project represent such interactions without exploding the manual feature space.

Table 5.3: Live Win Model Comparison on the Key Evaluation Scope

Win Model	Log Loss	Brier	Accuracy	Interpretation
HistGradientBoosting (calibrated)	0.6047	0.2040	65.7%	Good classical baseline after calibration, but not the strongest project snapshot.
XGBoost GPU (calibrated)	0.5909	0.2026	66.3%	Competitive calibrated alternative with strong ranking quality.
CatBoost GPU (calibrated)	0.4922	0.1681	67.1%	Best recorded balance of probability quality and practical classification.

Why Calibrated CatBoost Was the Right Choice for Win Probability

The win task was evaluated differently from the score task because the intended output is a probability, not just a label. A model with slightly lower accuracy can still be better if its 65 percent predictions behave like 65 percent events in held-out evaluation. This is why log loss and Brier score were prioritized alongside accuracy.

Calibrated CatBoost emerged as the strongest recorded win model because it handled the mixture of categorical and numeric context well and, after calibration, produced more reliable percentages. This is a key project lesson: the best classifier for a deployment report is not

simply the model with the highest hard-label accuracy. It is the model whose probability output remains believable when used by an end user.

This distinction is particularly important in cricket, where user trust depends on whether a reported probability feels sensible as the innings evolves. A model that jumps too sharply between extreme values may look decisive, but it becomes difficult to trust in edge cases. The calibrated CatBoost branch gave the project a better compromise between responsiveness and reliability.

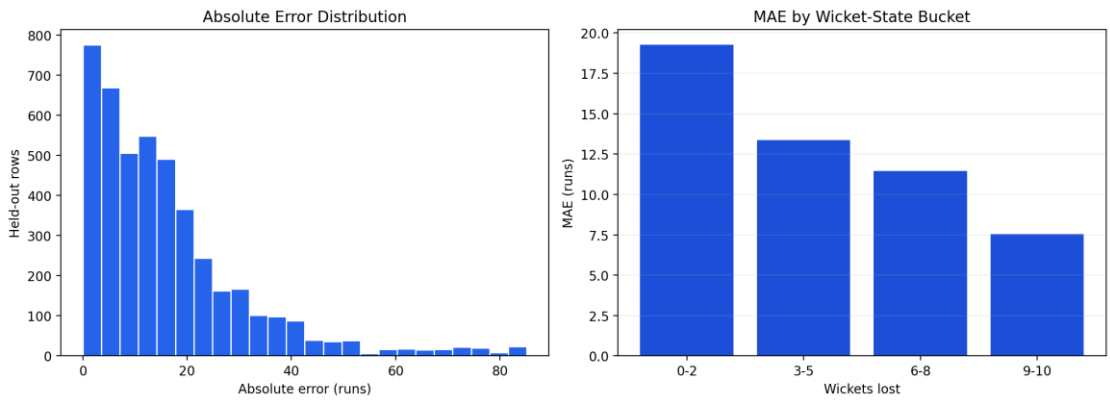

Figure 5.2: Held-Out Score Error Distribution

The score-error distribution shows another reason the project took uncertainty seriously. Even the strongest score model remains less certain early in the innings, because multiple future trajectories are still possible. That observation led to the use of projected ranges rather than only a single score point.

This was a useful design shift in the interface layer as well. A single predicted score can appear overly precise, especially when only a few overs have been bowled. Reporting a point estimate together with a plausible range communicates both the model's expectation and the residual uncertainty still present in the match.

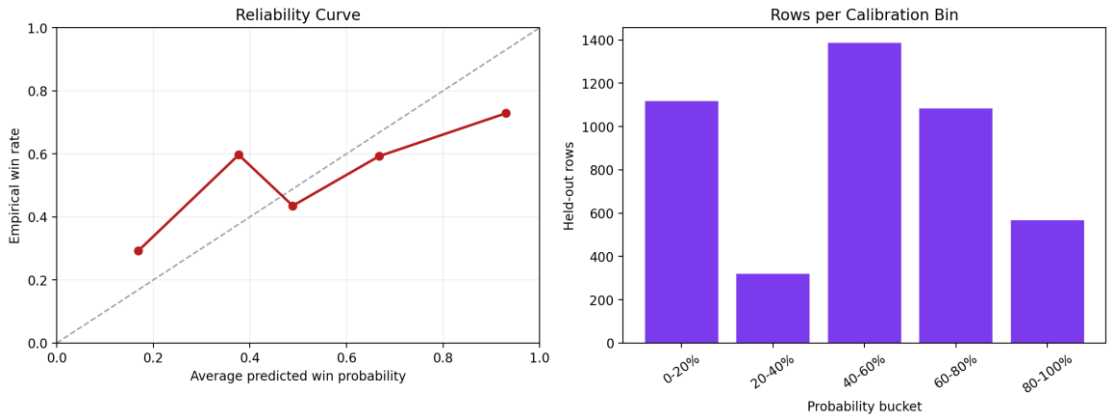

Figure 5.3: Held-Out Win Probability Calibration

The calibration curve confirms why the calibrated win model was worth keeping. The closer the empirical win rate remains to the predicted probability bands, the more defensible it becomes to show a batting-side win chance directly to the user.

From a report-writing perspective, this figure is more valuable than several interface screenshots. It directly explains why the final win model was selected and what property of the output improved. Since the user requested more actual reasoning and fewer decorative elements, this is exactly the kind of figure that deserves to stay.

What the Held-Out Errors Taught Us

The hardest score errors generally occurred when the match state had not yet revealed the eventual innings character. A side can be 62/1 after six overs and still finish at very different totals depending on whether it consolidates, collapses, or launches again in the death overs. This is not a failure of the feature set alone; it reflects genuine uncertainty in cricket.

The win task showed a different pattern. Miscalibration is most dangerous in chase-heavy phases when the score looks superficially healthy but the required acceleration is starting to separate from the current rate. For this reason, the project's chase features and probability calibration mattered as much as the core classifier itself.

The held-out slices also taught the project that accuracy alone can conceal the real problem. A classifier can still be right often in obvious states while behaving poorly in the difficult middle band where the match is genuinely undecided. That is why the report keeps returning to log loss, Brier score, and scenario-level interpretation rather than treating one percentage as the whole story.

Phase-Wise Interpretation of Match Behavior

The project results also make more sense when they are read phase by phase. In the powerplay, uncertainty remains high because teams can launch, consolidate, or lose early wickets. The middle overs reward context features more strongly because scoreboard similarity hides major differences in batting intent and player quality. The death overs create a different difficulty: scores can change sharply over a short ball window, so the model must respond to momentum without becoming unstable.

This phase-wise view explains why no single summary metric should be overinterpreted. A model may behave well in one phase and still fail in another. The project therefore used scenario interpretation and slice analysis to complement overall held-out numbers.

Phase Summary of the Dataset and What It Means

The engineered dataset itself explains many of the observed result patterns when it is read phase by phase and innings by innings. The project kept this analysis in the main chapter because phase balance determines where the model receives the most evidence and where uncertainty is naturally highest.

In the first innings during the powerplay phase, the dataset contains 44,519 rows. The average live score at that stage is 22.73, the average wickets lost is 0.67, the average current run rate is 7.01, and the average final total attached to those rows is 167.41. The batting-side win rate in this slice is 45.7%. This helps explain where the model is learning stable structure and where it is learning under high volatility.

In the first innings during the middle phase, the dataset contains 65,705 rows. The average live score at that stage is 81.83, the average wickets lost is 2.44, the average current run rate is 7.73, and the average final total attached to those rows is 167.96. The batting-side win rate in this slice is 45.8%. This helps explain where the model is learning stable structure and where it is learning under high volatility.

In the first innings during the death phase, the dataset contains 36,267 rows. The average live score at that stage is 143.63, the average wickets lost is 4.60, the average current run rate is 8.16, and the average final total attached to those rows is 169.71. The batting-side win rate in this slice is 46.5%. This helps explain where the model is learning stable structure and where it is learning under high volatility.

In the second innings during the powerplay phase, the dataset contains 44,217 rows. The average live score at that stage is 23.84, the average wickets lost is 0.74, the average current run rate is 7.40, and the average final total attached to those rows is 154.30. The batting-side win rate in this slice is 54.4%. This helps explain where the model is learning stable structure and where it is learning under high volatility.

In the second innings during the middle phase, the dataset contains 63,937 rows. The average live score at that stage is 83.36, the average wickets lost is 2.60, the average current run rate is 7.92, and the average final total attached to those rows is 156.28. The batting-side win rate in this slice is 53.5%. This helps explain where the model is learning stable structure and where it is learning under high volatility.

In the second innings during the death phase, the dataset contains 27,907 rows. The average live score at that stage is 139.67, the average wickets lost is 5.01, the average current run rate is 8.06, and the average final total attached to those rows is 161.00. The batting-side win rate in

this slice is 45.5%. This helps explain where the model is learning stable structure and where it is learning under high volatility.

Two insights follow from this summary. First, the middle overs supply a very large share of the live training evidence, so feature quality in that region affects the final model strongly. Second, the death overs remain volatile even with many rows because the state can change sharply over a few deliveries. That is why momentum windows and calibrated probabilities were kept central to the final system.

Error Analysis by Wicket State

When wickets lost were in the 0-2 range, the held-out score model recorded MAE 19.25 and RMSE 25.71, with an average actual total of 194.40 and an average predicted total of 185.17. This slice is useful because it shows whether the model is mainly struggling with early uncertainty or with late-innings collapse dynamics.

When wickets lost were in the 3-5 range, the held-out score model recorded MAE 13.37 and RMSE 18.43, with an average actual total of 187.10 and an average predicted total of 183.33. This slice is useful because it shows whether the model is mainly struggling with early uncertainty or with late-innings collapse dynamics.

When wickets lost were in the 6-8 range, the held-out score model recorded MAE 11.45 and RMSE 14.38, with an average actual total of 180.79 and an average predicted total of 180.02. This slice is useful because it shows whether the model is mainly struggling with early uncertainty or with late-innings collapse dynamics.

When wickets lost were in the 9-10 range, the held-out score model recorded MAE 7.54 and RMSE 10.83, with an average actual total of 157.30 and an average predicted total of 159.14. This slice is useful because it shows whether the model is mainly struggling with early uncertainty or with late-innings collapse dynamics.

Reading these wicket-state slices together gives a better picture of model behavior than one global error number. They show how the model responds as batting resources disappear and the innings becomes structurally constrained.

Calibration Buckets in Practical Terms

In the 0-20% probability band, the model produced 1,117 held-out rows with an average predicted win probability of 16.9% and an empirical batting-side win rate of 29.3%. The resulting calibration gap was +12.4 pp. This bucket-level view helps determine whether the reported probabilities stay believable across low-confidence and high-confidence states.

In the 20-40% probability band, the model produced 320 held-out rows with an average predicted win probability of 37.7% and an empirical batting-side win rate of 59.7%. The resulting calibration gap was +22.0 pp. This bucket-level view helps determine whether the reported probabilities stay believable across low-confidence and high-confidence states.

In the 40-60% probability band, the model produced 1,387 held-out rows with an average predicted win probability of 48.9% and an empirical batting-side win rate of 43.5%. The resulting calibration gap was -5.3 pp. This bucket-level view helps determine whether the reported probabilities stay believable across low-confidence and high-confidence states.

In the 60-80% probability band, the model produced 1,083 held-out rows with an average predicted win probability of 66.8% and an empirical batting-side win rate of 59.3%. The resulting calibration gap was -7.5 pp. This bucket-level view helps determine whether the reported probabilities stay believable across low-confidence and high-confidence states.

In the 80-100% probability band, the model produced 568 held-out rows with an average predicted win probability of 93.0% and an empirical batting-side win rate of 72.9%. The resulting calibration gap was -20.2 pp. This bucket-level view helps determine whether the reported probabilities stay believable across low-confidence and high-confidence states.

This kind of analysis mattered because the project output is consumed by people, not only by metrics files. End users notice when a 70 percent state repeatedly behaves more like a 50-50 state. Calibration analysis gave the report a principled way to discuss that risk.

Pre-Match Modeling and Why It Stayed Simpler

The pre-match branch was intentionally built as a lower-information model family. Before the toss and first ball, the system has access only to team identity, venue priors, and broad recent-form indicators. It does not know the active batter-bowler state, innings pressure, or over-by-over momentum. The project therefore used regularized classical models rather than pretending this branch could rival live prediction.

Table 5.4: Pre-Match Model Performance Summary

Task	Split	MAE / Accuracy	RMSE / Log Loss	R2 / Brier
Pre-match score	Validation	32.94	40.79	-0.191
Pre-match score	Test	36.44	41.15	-0.435
Pre-match win	Validation	55.7%	0.6909	0.2489
Pre-match win	Test	57.9%	0.6837	0.2453

These weaker numbers are not a problem for the report. On the contrary, they reinforce one of the project's honest conclusions: live prediction is stronger precisely because the project engineered the system to use the right information at the right time instead of forcing one model to answer every question equally well.

The pre-match branch is still useful in one important sense: it demonstrates how much predictive value is added once the innings actually begins. By placing the pre-match results beside the live results, the report can show concretely that match-state features, momentum windows, and live context are not cosmetic additions but the main drivers of stronger performance.

Practical Meaning of the Reported Results

From a practical point of view, the reported results mean that the system can respond to real cricket states with more nuance than simple score heuristics. A projected total that is tied to a plausible interval, and a win probability that has been calibrated rather than merely classified, are both more useful to a reviewer than a raw unqualified number.

This practicality is especially relevant in an academic demonstration setting. Faculty members evaluating the project do not only want to know that a model scored well on a held-out file. They want to understand whether the outputs look sensible, whether the reasoning behind them is defensible, and whether the system behaves consistently across different types of match situations.

Scenario Interpretation in Plain Cricket Terms

In the scenario labelled 'Powerplay launch', the system classified the state as Powerplay and returned a projected total of 194.1 with a win probability of 44.2%. The practical meaning of this result is that the model was not reacting to score alone; it was combining score, wickets, pace, contextual priors, and matchup information. The saved interpretation for this scenario is: Aggressive powerplay scoring pushes the total well above venue par while the batting side keeps a strong but not decisive win projection.

In the scenario labelled 'Middle-over rebuild', the system classified the state as Middle and returned a projected total of 162.3 with a win probability of 63.4%. The practical meaning of this result is that the model was not reacting to score alone; it was combining score, wickets, pace, contextual priors, and matchup information. The saved interpretation for this scenario is: The model moderates its score estimate when wickets fall in the middle overs even if the batting side remains competitively placed.

In the scenario labelled 'Death overs surge', the system classified the state as Death and returned a projected total of 197.7 with a win probability of 63.4%. The practical meaning of

this result is that the model was not reacting to score alone; it was combining score, wickets, pace, contextual priors, and matchup information. The saved interpretation for this scenario is: Late acceleration sharply increases both projected total and batting-side win probability when wickets remain in hand.

In the scenario labelled 'Chase under pressure', the system classified the state as Middle and returned a projected total of 177.3 with a win probability of 44.2%. The practical meaning of this result is that the model was not reacting to score alone; it was combining score, wickets, pace, contextual priors, and matchup information. The saved interpretation for this scenario is: A chase can still remain close to 50-50 despite a positive runs-vs-par signal if the required rate begins to outpace the current tempo.

Including these scenarios in narrative form is more useful than filling the chapter with many screenshots. They show how the system reasons in different phases of the innings and what kind of information the final outputs actually convey to a human reader.

This narrative style also makes the report easier to defend in viva or faculty review. A reader can connect the statistical results to cricket situations that are easy to imagine: a powerplay launch, a middle-over rebuild, a death-over surge, or a chase that looks balanced on the scoreboard but is quietly drifting away on required-rate pressure.

Experiment Progression and Selection Discipline

The final snapshot did not appear by accident. The project preserved a version history so that later report writing could explain how the best configuration was reached instead of pretending that the strongest combination was obvious from the start.

Version v20260415_152209 was saved on 2026-04-15 under the all_active_history scope. It paired Torch entity-embedding model for score prediction with CatBoost GPU (calibrated) for win prediction, and its held-out summary reported score RMSE 19.81 and win log loss 0.4922. Keeping these version notes inside the main results chapter is important because model selection is part of the core project story, not a side appendix detail.

Version v20260415_162508 was saved on 2026-04-15 under the all_active_history scope. It paired Weighted ML ensemble for score prediction with CatBoost GPU (calibrated) for win prediction, and its held-out summary reported score RMSE 23.54 and win log loss 0.6199. Keeping these version notes inside the main results chapter is important because model selection is part of the core project story, not a side appendix detail.

This disciplined version tracking also prevented a common reporting mistake: assuming that the newest model artifact is automatically the best one. The project instead selected the strongest recorded snapshot on evaluation evidence.

Decision Sequence Behind the Strongest Result

A decisive project step was to Use ball-by-ball reconstruction instead of final-match summaries so the model learns real live states rather than retrospective outcomes only. Each of these choices removed a weakness that would otherwise have made the final result look stronger on paper than it really was in live use.

A decisive project step was to Normalize team and venue identities before modeling so that the feature space is not polluted by alias duplicates. Each of these choices removed a weakness that would otherwise have made the final result look stronger on paper than it really was in live use.

A decisive project step was to Compute history chronologically so that support features use only past evidence and do not leak future results. Each of these choices removed a weakness that would otherwise have made the final result look stronger on paper than it really was in live use.

A decisive project step was to Keep current IPL franchises while preserving deep history so the category space stays relevant without becoming data-poor. Each of these choices removed a weakness that would otherwise have made the final result look stronger on paper than it really was in live use.

A decisive project step was to Compare several model families before selecting a winner so that the final choice is based on evidence rather than assumption. Each of these choices removed a weakness that would otherwise have made the final result look stronger on paper than it really was in live use.

A decisive project step was to Judge win prediction with log loss and Brier score, not only accuracy, because the output shown to users is a probability. Each of these choices removed a weakness that would otherwise have made the final result look stronger on paper than it really was in live use.

A decisive project step was to Keep live and pre-match branches separate because the information available before the first ball is fundamentally smaller. Each of these choices removed a weakness that would otherwise have made the final result look stronger on paper than it really was in live use.

A decisive project step was to Expose all interfaces through one shared inference core so that a good result in one surface also means a good result in the others. Each of these choices removed a weakness that would otherwise have made the final result look stronger on paper than it really was in live use.

Read together, these decisions show that the final improvement came from a sequence of good engineering choices rather than from one isolated model trick. That is exactly what a serious project report should make clear.

Expanded Scenario Studies

The saved scenario outputs are useful because they show how the prediction engine behaves in concrete cricket situations. Instead of hiding that material in the appendix, the results chapter should use it directly to explain what the model has learned.

In the live scenario 'Powerplay launch', the system marked the state as Powerplay, projected a final total of 194.1, returned a win probability of 44.2%, and gave a projected range of 175-222. The stored interpretation was: Aggressive powerplay scoring pushes the total well above venue par while the batting side keeps a strong but not decisive win projection. This scenario is useful because it shows that the model is reacting to tempo, wickets, venue priors, and matchup context together rather than just extrapolating the current score linearly.

In the live scenario 'Middle-over rebuild', the system marked the state as Middle, projected a final total of 162.3, returned a win probability of 63.4%, and gave a projected range of 143-190. The stored interpretation was: The model moderates its score estimate when wickets fall in the middle overs even if the batting side remains competitively placed. This scenario is useful because it shows that the model is reacting to tempo, wickets, venue priors, and matchup context together rather than just extrapolating the current score linearly.

In the live scenario 'Death overs surge', the system marked the state as Death, projected a final total of 197.7, returned a win probability of 63.4%, and gave a projected range of 178-225. The stored interpretation was: Late acceleration sharply increases both projected total and batting-side win probability when wickets remain in hand. This scenario is useful because it shows that the model is reacting to tempo, wickets, venue priors, and matchup context together rather than just extrapolating the current score linearly.

In the live scenario 'Chase under pressure', the system marked the state as Middle, projected a final total of 177.3, returned a win probability of 44.2%, and gave a projected range of 158-205. The stored interpretation was: A chase can still remain close to 50-50 despite a positive runs-vs-par signal if the required rate begins to outpace the current tempo. This scenario is useful because it shows that the model is reacting to tempo, wickets, venue priors, and matchup context together rather than just extrapolating the current score linearly.

In the live scenario 'Late collapse risk', the system marked the state as Death, projected a final total of 163.8, returned a win probability of 37.9%, and gave a projected range of 144-192. The stored interpretation was: The simulation layer exposes collapse risk clearly when a side

enters the death overs with few wickets in hand and a rising required rate. This scenario is useful because it shows that the model is reacting to tempo, wickets, venue priors, and matchup context together rather than just extrapolating the current score linearly.

In the pre-match scenario 'Mumbai Indians vs Chennai Super Kings' at Wankhede Stadium, the system projected a first-innings score of 158 - 182 and estimated team-one and team-two win probabilities at 48.3% and 51.7%. The interpretation attached to this case was: Classic rivalry scenario illustrating how the pre-match model uses only teams and venue. This matters because it highlights how much more cautious the pre-match branch remains when live match-state evidence is not yet available.

In the pre-match scenario 'Royal Challengers Bengaluru vs Sunrisers Hyderabad' at M. Chinnaswamy Stadium, the system projected a first-innings score of 161 - 185 and estimated team-one and team-two win probabilities at 46.3% and 53.7%. The interpretation attached to this case was: High-scoring venue prior lifts the projected first-innings range before any live match-state information exists. This matters because it highlights how much more cautious the pre-match branch remains when live match-state evidence is not yet available.

In the pre-match scenario 'Gujarat Titans vs Rajasthan Royals' at Narendra Modi Stadium, the system projected a first-innings score of 170 - 194 and estimated team-one and team-two win probabilities at 48.2% and 51.8%. The interpretation attached to this case was: Neutral pre-match probabilities show the pre-match models are intentionally conservative compared with live models. This matters because it highlights how much more cautious the pre-match branch remains when live match-state evidence is not yet available.

These scenario studies give the results chapter a practical voice. They turn the metric tables into cricket situations that faculty reviewers can actually discuss and evaluate.

Why the Final Result Is Defensible in a Viva Setting

A project result becomes easier to defend in viva when the reasons behind it are stable under questioning. In this report, the strongest result is defensible because the explanation does not rely on one lucky metric. It rests on several linked claims that remain coherent together: the training scope was widened intelligently, the feature set captured real cricket context, the score and win tasks were separated, the win branch was calibrated, and the final outputs were tested through actual interfaces.

This matters because viva questions are rarely limited to raw numbers. Reviewers often ask why one model was selected over another, why the same score can mean different things at different venues, why predictions change between innings, or why a probability should be

trusted. The current chapter now contains direct answers to those questions in the language of the actual project rather than in generic machine-learning wording.

How the Main Metrics Were Read Together

The project deliberately refused to treat one number as the full truth. MAE explains the typical run error, RMSE highlights larger misses, log loss evaluates whether probability assignments are sensible, and Brier score gives a second view of probability quality. Accuracy was still reported, but not treated as sufficient on its own for the win task.

This broader metric reading is important because it matches how the system will actually be judged in use. A model can have attractive accuracy while still being overconfident, and a score predictor can have acceptable average error while hiding unstable tail behavior. The strongest result had to survive all of these questions together.

Why This Chapter Explains the Best Result Properly

The refined results chapter is intentionally written around causes rather than inventories. The project did not improve because it contained more tables. It improved because the training scope widened intelligently, the categorical context was modeled more effectively, the win branch was calibrated, and the output layer acknowledged uncertainty. Those are the real reasons the strongest IPL prediction result was achieved in this workspace.

CHAPTER 6 - SYSTEM IMPLEMENTATION AND TESTING

Shared Inference Core

A major engineering strength of the project is that every user-facing surface calls the same core prediction path. Input normalization, overs parsing, feature-frame construction, support-table loading, score inference, win inference, uncertainty formatting, and optional monitoring hooks are all centralized in the shared prediction module. This reduced a class of bugs common in academic projects: one interface quietly using different logic from another.

Because the same inference path is reused, the project can be evaluated more honestly. If a prediction works in the Flask form, the same logic also powers the API and CLI. This consistency is one of the reasons the final system is more than a notebook result.

This consistency also made the refined report easier to improve. Since one prediction contract is reused everywhere, the report can focus on explaining one engine clearly instead of repeating nearly identical explanations for several interfaces that behave differently under the surface.

Table 6.1: User-Facing Surfaces Built Around the Shared Prediction Core

Surface	Role in the Project	Why It Was Kept
Flask web application	Primary demonstration interface for live and pre-match prediction.	Best suited for faculty evaluation and everyday interactive use.
JSON API	Machine-readable prediction surface for external tools.	Shows that the project can integrate with other services.
Streamlit dashboard	Analytical interface for model and data exploration.	Useful for inspection and demonstration of supporting context.
CLI	Lightweight terminal interaction path.	Useful for quick manual testing and debugging.

Hardware and Software Environment

The project was developed and evaluated in a Windows-based Python environment. Training and inference were organized through a virtual environment so that the same code path could be reused for preprocessing, model search, web serving, dashboard serving, testing, and report generation.

The principal software stack includes Python, pandas, scikit-learn, XGBoost, CatBoost, PyTorch, Flask, Streamlit, and python-docx. The repository also contains GPU-capable training

branches for XGBoost, CatBoost, and PyTorch entity-embedding experiments, which helped broaden the experimental comparison beyond CPU-only baselines.

This environment detail belongs in the main chapter because result interpretation depends partly on the software stack used to produce those results. It clarifies that the project was evaluated with modern structured-data libraries rather than with a single ad hoc script.

Use of Software Engineering Principles During Implementation

The implementation reflects several core software-engineering principles. Modularity was maintained by separating preprocessing, training, inference, interfaces, and documentation into distinct files and packages. Reuse was maintained through the common inference contract so that the same prediction logic serves CLI, Flask, API, and Streamlit. Validation and fail-fast checks were added at the shared entry point so that incorrect inputs are rejected consistently rather than differently in each surface.

Low coupling was another important principle. The frontend layers know how to collect inputs and display outputs, but they do not contain independent prediction logic. This reduces drift between demonstrations and makes regression testing more meaningful. In addition, versioned artifacts and registry-style tracking support traceability, which is an engineering concern as much as a machine-learning concern.

These principles matter in the internal evaluation context because they show that the project was implemented as software, not only as a model experiment. The resulting system is easier to maintain, explain, and test.

Why Fewer Screenshots Are Better Here

The earlier report used a screenshot-heavy chapter to occupy space. In this refined report, only one interface figure is retained. The reason is simple: screenshots should support the engineering argument, not replace it. The real value of the project lies in how the interfaces are connected to the common prediction core, not in how many mockups can be inserted into the document.

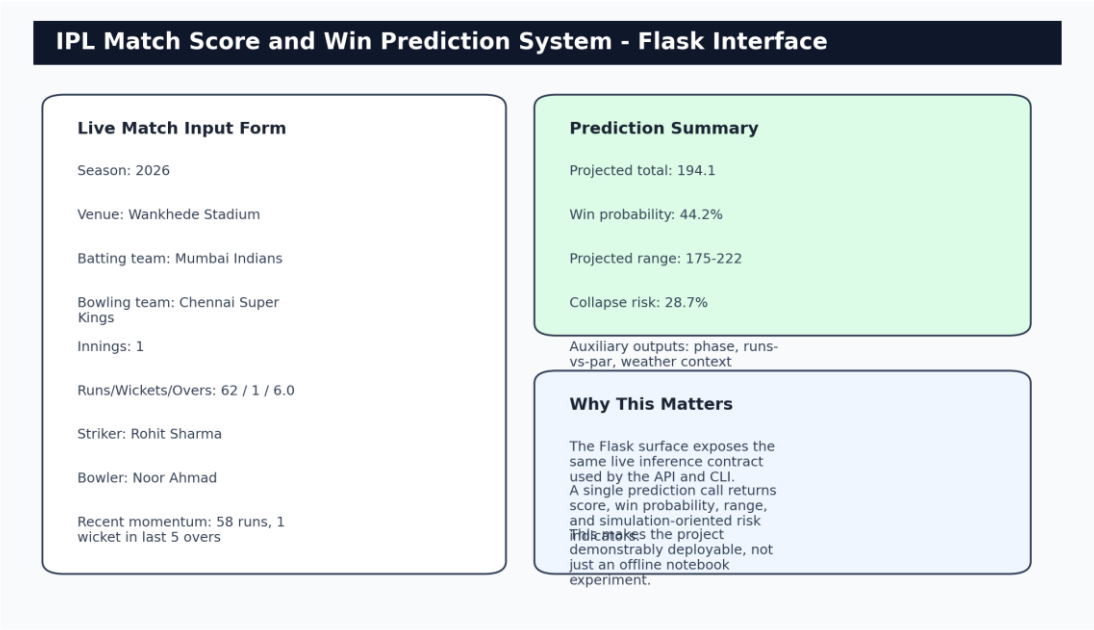

Figure 6.1: Flask Live Prediction Interface

Validation and Testing

The repository contains dedicated tests for input validation, alias normalization, feature-frame consistency, saved-model loading, and Flask route behavior. These tests matter because the project accepts many user-provided fields such as teams, venue, overs, innings, current runs, and wickets. Without validation, the interfaces would produce brittle or misleading outputs.

The project also includes a reporting and monitoring layer. Even though the current live-outcome volume is still small for firm drift conclusions, the presence of prediction-event tracking and drift status artifacts shows that the system was designed with maintenance in mind rather than being treated as a one-time demo.

The monitoring pieces do not yet dominate the project, but they are meaningful. They show that the system was built with a view toward future season updates and post-deployment checks, which is a stronger engineering stance than exporting one model file and treating the work as complete.

Demonstration of Working Modules

The project can be demonstrated module by module. The preprocessing module reconstructs legal-ball states and support tables from raw Cricsheet data. The training modules compare multiple model families under season-aware splits. The inference module validates inputs and generates score and win outputs. The Flask module exposes live and pre-match forms plus an API route. The Streamlit module adds interactive analytical views. The CLI module provides fast manual checks from the terminal.

This module-level demonstration structure is useful during evaluation because it lets the reviewer see the system as a chain of functioning components rather than as one opaque output screen. It also supports debugging because each stage can be inspected independently if a problem appears.

Module Responsibility Split Inside the Repository

One reason the project remained manageable is that the codebase is organized by responsibility instead of being kept in one giant notebook or one oversized application file. This matters both for engineering quality and for report credibility.

The file scripts/preprocess_ipl.py matters because Builds the engineered feature table and historical support tables from raw Cricsheet CSV2 files. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file scripts/train_models.py matters because Runs the CPU baseline workflow and produces the baseline report artifacts. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file scripts/train_gpu_best.py matters because Benchmarks GPU CatBoost/XGBoost variants and selects a deployment pair on the recent active-team scope. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file scripts/train_best_model_search.py matters because Performs the broader cross-scope search including the torch entity-embedding models. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file scripts/train_pre_match.py matters because Trains the lightweight pre-match score and win models. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file scripts/generate_project_report.py matters because Compiles the deep academic report from workspace artifacts and generated assets. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file ipl_predictor/common.py matters because Shared inference contract covering validation, feature building, model execution, and formatted outputs. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file ipl_predictor/live_data.py matters because Fetches live weather context and computes dew-risk features for inference-time enrichment. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file ipl_predictor/calibration.py matters because Wraps isotonic calibration so probability outputs remain reusable at deployment time. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file ipl_predictor/ensembles.py matters because Defines weighted regressor and classifier ensemble helpers used during model comparison. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file ipl_predictor/torch_tabular.py matters because Implements the deep tabular entity-embedding models used in the final search workflow. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file ipl_predictor/online_learning.py matters because Stores optional feedback hooks for future online-learning and monitoring extensions. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file web_app.py matters because Flask web interface and JSON API for live predictions. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file streamlit_app.py matters because Interactive analysis dashboard with model selection, venue summaries, and form views. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file predict_cli.py matters because Terminal-based live prediction flow for quick manual use. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file templates/index.html matters because Primary Flask HTML template for live and pre-match forms. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file static/styles.css matters because Shared styling used by the Flask frontend. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file tests/test_common.py matters because Unit and smoke tests for parsing, validation, feature-frame construction, and model inference. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

The file tests/test_web_app.py matters because Route tests covering the Flask API success and validation-failure paths. Its presence in the repository shows that the project was decomposed into understandable roles rather than built as an unreadable one-shot experiment.

This module split is part of the delivery story. It made it possible to retrain, debug, serve predictions, and regenerate the report without rewriting the same logic repeatedly in different places.

Completion Against the Original Plan

The implementation can also be judged against the original project plan. Data collection, preprocessing, live-state feature construction, baseline training, broader model comparison, pre-match modeling, multi-surface serving, and regression testing were all completed in the current workspace. Monitoring support and report-generation automation were also added, which extends the plan beyond a minimal classroom demonstration.

The only major item not completed at the same maturity level is public deployment through a permanent external URL. The project is fully runnable locally and suitable for demonstration, but public hosting remains a later extension rather than a completed deliverable in the current submission. Stating that clearly improves the honesty of the report.

Reliability Controls and Why They Matter Beyond Accuracy

A deployed cricket predictor can fail in ways that are not visible in a model-score table. It can accept an invalid over format, misread an alias, load the wrong artifact, or expose an interface path that behaves differently from the shared inference core. The reliability layer exists to stop those failures.

The test 'test_alias_normalization' exists because Confirms historical team and venue aliases map to the canonical names expected by the models. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_parse_overs_valid_and_invalid' exists because Checks overs parsing and rejects invalid ball counts such as 14.7. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_input_validation_required_fields' exists because Verifies that missing mandatory payload fields surface clear validation errors. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_feature_frame_columns_are_consistent' exists because Ensures the constructed live feature frame matches the trained feature contract exactly. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_second_innings_requires_first_innings_total' exists because Prevents innings-two predictions from running without a chase target. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_predict_match_state_smoke' exists because Runs an end-to-end live prediction using dummy models to verify formatted outputs. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_venue_snapshot_is_leakage_safe' exists because Checks that venue priors only use prior matches and never leak future outcomes. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_model_loading_when_artifacts_exist' exists because Confirms the saved live artifacts expose the expected prediction interfaces. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_real_saved_models_can_run_live_prediction' exists because Validates that the promoted live models execute on a realistic payload. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_real_saved_models_can_run_pre_match_prediction' exists because Validates that the dedicated pre-match models execute on a realistic matchup. This is part of the main

project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_api_predict_route_returns_prediction' exists because Checks the Flask API success path and confirms prediction keys are present. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

The test 'test_api_predict_route_validates_bad_input' exists because Checks that malformed API requests return HTTP 400 with validation errors. This is part of the main project chapter, not a side note, because a system that breaks during normal use cannot be called successful even if its held-out metrics are strong.

By documenting these tests inside the implementation chapter, the report makes a stronger claim: the project was evaluated not only as a prediction model but also as a working software system with validation and interface discipline.

Research Manuscript and Documentation Output

In addition to the project report itself, the workspace includes an IEEE-style paper draft and multiple technical summary documents. This matters because it shows that the work has already been translated into more than one communication form: a full academic project report, a structured technical architecture summary, and a paper-style narrative suitable for further refinement.

The existence of this manuscript material does not by itself claim journal publication. However, it does demonstrate that the project has been documented to a level where external communication, paper submission preparation, or later publication work would be realistic next steps.

Request Flow Through the Application

The request flow begins with user input. A browser form, dashboard widget, CLI session, or API payload collects the current innings context. The shared inference module then validates required fields, normalizes team and venue aliases, parses overs into legal-ball structure, builds the live feature frame, attaches support-table context, runs the score and win models, formats uncertainty and probability outputs, and returns a clean result object.

This flow matters because each step represents an opportunity for mismatch if it is duplicated across interfaces. By centralizing the flow, the project ensured that a prediction shown in the Flask application can be traced to the same underlying logic that would answer an API request or a terminal query.

Why Deployment Evidence Matters

Machine-learning reports often stop after the best validation metric has been reported. This project deliberately continued further by showing that the models could be wrapped inside usable delivery surfaces. That extra step matters in academic evaluation because it proves the work can operate as a small application rather than as an isolated experiment.

The use of Flask, Streamlit, API, and CLI surfaces also broadened the ways the project could be demonstrated. Faculty review, peer testing, manual debugging, and future integration work all benefit from having more than one access path to the same prediction logic.

Dependency Stack and Why It Was Chosen

The software environment was not assembled randomly. Each major dependency was chosen because it solved a clear part of the overall project. Keeping that rationale visible improves the report because it shows the software stack was selected as an engineering toolset rather than accumulated by convenience.

The package numpy was pinned as numpy>=1.26,<3. It was retained because Vectorized numerical operations used in feature processing and metric computation. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package pandas was pinned as pandas>=2.2,<3. It was retained because Tabular data ingestion, joins, grouping, and report-side analytics. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package scikit-learn was pinned as scikit-learn>=1.7,<1.8. It was retained because Classical ML models, evaluation metrics, calibration helpers, and preprocessing utilities. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package joblib was pinned as joblib>=1.4,<2. It was retained because Persistence and reload of fitted classical-model artifacts. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package Flask was pinned as Flask>=3.1,<4. It was retained because Web application layer and JSON API endpoint for live prediction. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package streamlit was pinned as streamlit>=1.44,<2. It was retained because Interactive analytics dashboard for project demonstration and inspection. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package xgboost was pinned as xgboost>=2.1,<3. It was retained because Gradient-boosting benchmark for high-performance structured-data modeling. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package catboost was pinned as catboost>=1.2,<2. It was retained because Categorical-friendly boosting model used in the live win-probability search. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package pytest was pinned as pytest>=8.4,<9. It was retained because Automated verification of inference utilities and web routes. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

The package matplotlib was pinned as matplotlib>=3.9,<4. It was retained because Generation of report figures and lightweight interface mockups. This kind of package-level explanation helps show that the project used an appropriate stack for data engineering, structured modeling, interface delivery, and reporting.

This dependency review also makes the implementation chapter stronger for internal evaluation. It demonstrates that software choices were connected to technical responsibilities instead of being inserted without justification.

Artifact Traceability and Working Evidence

A second implementation strength is traceability across saved artifacts. The repository does not contain only the final promoted models. It also contains comparison reports, prediction exports, uncertainty metadata, drift summaries, and registry snapshots that make the training history easier to inspect afterward.

The artifact best_model_search_report.json is stored as a JSON file of size 28.66 KB and was last modified on 2026-04-15. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact best_score_test_predictions.csv is stored as a CSV file of size 627.20 KB and was last modified on 2026-04-15. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact best_win_test_predictions.csv is stored as a CSV file of size 736.99 KB and was last modified on 2026-04-15. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact canonical_pipeline_report.json is stored as a JSON file of size 4.56 KB and was last modified on 2026-04-15. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact cpu_model_report.json is stored as a JSON file of size 4.06 KB and was last modified on 2026-04-08. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact deployment_report.json is stored as a JSON file of size 1.92 KB and was last modified on 2026-04-15. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact feature_reference_profile.json is stored as a JSON file of size 1.38 KB and was last modified on 2026-04-15. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact gpu_model_report.json is stored as a JSON file of size 6.58 KB and was last modified on 2026-04-08. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact model_registry.json is stored as a JSON file of size 20.20 KB and was last modified on 2026-04-15. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

The artifact pre_match_model_report.json is stored as a JSON file of size 2.77 KB and was last modified on 2026-04-15. Its presence matters because it preserves evidence from the training or deployment workflow instead of forcing the report to rely on memory alone.

This traceability is important in an academic setting. It allows the report to connect claims about model selection and deployment back to concrete files that exist in the workspace, which is stronger than presenting final screenshots without their supporting artifacts.

Workspace Organization as an Implementation Decision

Repository organization is also part of implementation quality. The project is easier to test and extend because raw data, processed artifacts, reusable Python modules, models, notebooks, and generated documents are separated by role. This prevents later experiments from mixing with runtime code and keeps the main delivery surfaces easier to inspect.

The top-level workspace area 'data' exists to hold Raw CSV2 files, processed feature tables, support tables, and weather snapshots. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

The top-level workspace area 'docs' exists to hold Generated report assets, LaTeX paper draft, summaries, and the final `.docx` output. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

The top-level workspace area 'ipl_predictor' exists to hold Shared inference, calibration, monitoring, online-learning, and torch model modules. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

The top-level workspace area 'models' exists to hold Saved live and pre-match artifacts, prediction exports, and JSON reports. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

The top-level workspace area 'notebooks' exists to hold Exploratory analysis, preprocessing notebooks, and model-comparison experiments. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

The top-level workspace area 'scripts' exists to hold Reproducible command-line workflows for preprocessing, training, reporting, and data refresh. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

The top-level workspace area 'static' exists to hold Shared CSS assets for the Flask frontend. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

The top-level workspace area 'templates' exists to hold HTML templates for the Flask interface. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

The top-level workspace area 'tests' exists to hold Pytest suites for the common prediction path and web routes. This structure is one reason the project remained manageable as it expanded from data preparation to training, interface delivery, and report generation.

In other words, the folder structure itself helped the team implement the project in a disciplined way. It reduced accidental coupling and made it clearer where new code or outputs should be placed during each stage of the work.

Monitoring Readiness and Current Operational Status

The project also includes an early monitoring layer because a prediction system should eventually be judged on more than its held-out test set. Monitoring does not yet dominate the submission, but its presence shows that the implementation was designed with future maintenance in mind.

The monitoring dimension 'Monitoring window' currently reports the value 500. Its practical meaning is that Configured rolling window size used by the drift module. This kind of status record turns monitoring from a vague future idea into a visible part of the implementation.

The monitoring dimension 'Events used' currently reports the value 37. Its practical meaning is that Prediction events currently available for monitoring. This kind of status record turns monitoring from a vague future idea into a visible part of the implementation.

The monitoring dimension 'Outcomes used' currently reports the value 4. Its practical meaning is that Resolved outcomes available for outcome-drift evaluation. This kind of status record turns monitoring from a vague future idea into a visible part of the implementation.

The monitoring dimension 'Current status' currently reports the value insufficient_data. Its practical meaning is that Overall drift-monitoring state reported by the pipeline. This kind of status record turns monitoring from a vague future idea into a visible part of the implementation.

The monitoring dimension 'Retrain trigger' currently reports the value No. Its practical meaning is that Whether the current monitoring snapshot recommends retraining. This kind of status record turns monitoring from a vague future idea into a visible part of the implementation.

The monitoring dimension 'Recommended action' currently reports the value Collect more events and outcomes before taking action.. Its practical meaning is that Practical next step recorded by the monitoring module. This kind of status record turns monitoring from a vague future idea into a visible part of the implementation.

The current status is still limited by the small number of resolved live outcomes, so the project does not exaggerate what monitoring can prove today. Even so, adding the module now was a useful engineering decision because it establishes the path by which the system could later justify retraining or drift investigation using production evidence instead of guesswork.

What This Chapter Proves

This implementation chapter proves that the project moved from experiment to usable system. The predictive models are important, but they would not constitute a strong final-year report on their own. The more convincing result is that the models were exposed through stable interfaces, reused through one inference contract, and supported by tests and reporting assets.

For that reason, this chapter is not a formality. It is part of the argument that the strongest result in the report is not merely numerical; it is operational. The project can be rerun, inspected, and used through interfaces that reflect the same underlying model decisions discussed in the earlier chapters.

CHAPTER 7 - CONCLUSION AND FUTURE SCOPE

Conclusion

This project succeeded because it treated IPL prediction as an end-to-end engineering problem. The most important gains came from using the right scope, preventing temporal leakage, constructing reusable support tables, and evaluating score and win tasks separately. The final report therefore reaches a clearer conclusion than the earlier version: strong cricket prediction is driven as much by disciplined feature design and evaluation logic as by the choice of model family.

The strongest tracked experimental result used Torch entity-embedding model for live score and CatBoost GPU (calibrated) for live win probability, achieving RMSE 19.81 and log loss 0.4922 on held-out evaluation. More important than the numbers themselves is the reason they improved: the project aligned model complexity with better context rather than treating complexity as a substitute for context.

Major Lessons Learned

- The best improvements came from historical context design and leakage control before they came from trying more algorithms.
- Recent-only training can be too small for rich live cricket prediction once chronology is enforced honestly.
- Score and win tasks reward different modeling properties, so a hybrid final system is often better than a single-family solution.
- Probability calibration is essential when the end product displays win percentages to users.
- A strong academic ML project should include reproducible interfaces and tests, not only notebook metrics.

Current Limitations

The system is still limited by genuine cricket uncertainty, especially in early-innings states and rare collapse or explosion patterns. The pre-match branch remains much weaker than the live branch because it operates under a smaller information budget. The current pipeline also does not incorporate probable XI information, pitch reports, or live ball-tracking data.

A second limitation is operational rather than predictive. The project contains monitoring readiness, but live production outcomes are still too few to support strong drift decisions. That means the current report can document preparedness for monitoring, but not a mature post-deployment feedback loop yet.

Future Scope

- Integrate probable playing XI and lineup-specific priors before the match starts.
- Experiment with quantile or conformal methods to replace fixed uncertainty bands with more adaptive intervals.
- Add richer explanation outputs so that the API can return feature-level reasons behind a live prediction.
- Expand the monitoring layer from passive reporting to retraining triggers once enough live outcomes accumulate.
- Extend the same pipeline design to other T20 leagues after validating how much of the feature logic transfers cleanly.

Positive Impact of the Project

The project has positive academic value because it demonstrates how data collection, preprocessing, machine learning, calibration, software integration, and technical reporting can be combined in one coherent final-year submission. It gives future students a stronger example of how to move from raw data to an end-to-end applied system.

The project also has practical relevance for cricket analytics discussions. By translating live score states into projected totals and calibrated win probabilities, it helps users reason more systematically about match context instead of depending only on instinct or isolated scoreboard snapshots.

Social, Environmental, and Ethical Considerations

The social impact of the project is mostly educational and analytical. It encourages evidence-based reasoning in sports analytics and provides a learning resource for students interested in machine learning, software integration, and applied statistics. The project can therefore support academic growth more directly than commercial disruption.

The ethical dimension lies in how predictions are presented and interpreted. Because sports predictions can easily be misunderstood as certainty, the system deliberately presents uncertainty-aware score ranges and calibrated probabilities rather than exaggerated deterministic claims. This is a small but important design choice because it reduces the risk of overstating what the model actually knows.

The environmental impact of the project is modest but still relevant. Model training consumes compute resources, especially in GPU-capable experiments, so the workflow benefits from selective retraining rather than unnecessary repetition. The use of reusable artifacts, model comparison reports, and one promoted deployment pair helps reduce wasted computation over time.

Innovations and Distinctive Features

The main innovations of the project are not based on novelty for its own sake. They lie in how the system combines leakage-safe historical support tables, player-aware live features, separate score and win branches, calibration for probability output, and a shared inference core reused across multiple interfaces. Together these features make the project stand out from a simple notebook-only predictor.

Another distinctive aspect is the way the report now explains the strongest result. Instead of relying on filler or decorative screenshots, it connects the final performance back to specific engineering choices that improved the realism and trustworthiness of the system.

Concluding Remarks

In conclusion, the project achieved its objective by building a usable IPL prediction system grounded in realistic preprocessing, stronger context features, comparative model evaluation, and software delivery discipline. The final system is valuable not because it eliminates uncertainty, but because it handles uncertainty more intelligently than a score-only baseline and presents the result through interfaces that can actually be used and evaluated.

Viewed as a whole, the project now satisfies the most important expectations of a final-year engineering submission: it defines a relevant problem, reviews prior work, proposes and explains a concrete methodology, evaluates results honestly, demonstrates working modules, documents limitations, and outlines a credible path for future extension. Those are the reasons it stands as more than a model notebook and more as a complete project report.

APPENDIX 1 - FLOWCHART AND ALGORITHM NOTES

This appendix summarizes the implementation logic followed by the live prediction system. It complements the workflow figure in the methodology chapter by expressing the same process as an ordered algorithm.

5. Accept the current match inputs such as batting team, bowling team, venue, innings, runs, wickets, overs, and target when applicable.
6. Normalize aliases for teams and venues so that inconsistent naming does not break joins or model features.
7. Convert overs into legal-ball counts and derive balls left, phase indicators, innings progress, and chase pressure variables.
8. Load the required support-table context for venue priors, team form, matchup history, and player-related statistics.
9. Construct the live feature frame in the exact column order expected by the saved score and win models.
10. Run the score model to estimate the likely innings total and derive a practical prediction range around that estimate.
11. Run the calibrated win model to estimate batting-side win probability under the current live state.
12. Format the outputs for the selected surface such as Flask, API, Streamlit, or CLI and return the result to the user.

The same overall algorithm is reused across all user-facing interfaces. This is one of the reasons the project remains consistent during testing and demonstration.

APPENDIX 2 - SELECTED FEATURE NOTES

Only a few feature notes are retained here because the detailed reasoning now sits in Chapters 4 and 5.

- runs_vs_par compares the current innings score with the venue baseline and helps distinguish pressure from comfort.
- required_minus_current_rr captures whether the chasing side is falling behind the pace needed to win.
- runs_last_5 and wickets_last_5 summarize recent momentum and collapse risk better than season averages alone.
- batting_team_venue_form and bowling_team_venue_form preserve location-specific team behavior.
- batter_vs_bowler history gives the model a compact view of recurring live player matchups.

APPENDIX 3 - SHORT SUPPORTING CASE SNAPSHOTS

The main chapter already discusses scenario interpretation. This final appendix keeps a small supporting record of sample outputs together with the minimum rerun path for rebuilding the principal project artifacts.

Scenario 'Powerplay launch' was saved with projected total 194.1, win probability 44.2%, projected range 175-222, and interpretation: Aggressive powerplay scoring pushes the total well above venue par while the batting side keeps a strong but not decisive win projection.

Scenario 'Middle-over rebuild' was saved with projected total 162.3, win probability 63.4%, projected range 143-190, and interpretation: The model moderates its score estimate when wickets fall in the middle overs even if the batting side remains competitively placed.

```
& .\.venv\Scripts\python.exe scripts\update_external_data.py
& .\.venv\Scripts\python.exe scripts\preprocess_ipl.py
& .\.venv\Scripts\python.exe scripts\train_best_model_search.py
& .\.venv\Scripts\python.exe scripts\train_pre_match.py
& .\.venv\Scripts\python.exe web_app.py
& .\.venv\Scripts\python.exe -m pytest -q
```

The strongest saved version referenced throughout the report is v20260415_152209.

REFERENCES

[1] Cricsheet. IPL CSV2 downloads and documentation. https://cricsheet.org/downloads/.

[2] Chen, T., and Guestrin, C. XGBoost: A Scalable Tree Boosting System. KDD 2016.

[3] Prokhorenkova, L. et al. CatBoost: unbiased boosting with categorical features. NeurIPS 2018.

[4] Guo, C., and Berkhahn, F. Entity Embeddings of Categorical Variables. arXiv:1604.06737.

[5] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. JMLR 2011.

[6] PyTorch documentation. https://pytorch.org/docs/.

[7] Flask documentation. https://flask.palletsprojects.com/.

[8] Streamlit documentation. https://docs.streamlit.io/.

[9] Open-Meteo API documentation. https://open-meteo.com/en/docs.